WikiOracle Documentation

Contents

WikiOracle **6**
- The Problem . . . 6
- What Makes WikiOracle Different . . . 6
  - Truth . . . 6
  - Data Soverignty . . . 6
  - Sharing Truth via QR Code . . . 6
  - Democracy . . . 7
  - Distribution . . . 7
- Current Prototype . . . 7
- Longer-Term Direction . . . 7
- How to Contribute . . . 7
- Getting Started . . . 7
- Documentation . . . 8
- Research Materials . . . 8

WikiOracle **9**
- Architectural Overview . . . 9
- Client . . . 9
- Server . . . 10
- Conversation . . . 10
- TruthSet . . . 11
- Example . . . 11

WikiOracle Constitution **11**
- Preamble . . . 11
- Core Commitments . . . 12
- Truth Primitives (What the System Is Allowed to Believe) . . . 12
- Plurality, Dispute, and Minority Preservation . . . 13
- Independence From Any Single AI Vendor . . . 13
- Authority Delegation Must Be Bounded and Secure . . . 13
- Local-First Data and Auditability . . . 13
- Safety as Freedom, Empathy, and Truth . . . 13

Installation **14**
- Environment and System Preparation . . . 14
  - Requirements . . . 14
  - Python dependencies . . . 14
  - Data and tokenizer bootstrap . . . 14
  - Environment variables . . . 14
- The Makefile: Running and Building . . . 14
  - Top-level targets . . . 14
  - Training (`train_*`) . . . 15

Test / Evaluation (`test_*`) . . . 15
Run / Inference (`run_*`) . . . 15
Sync (`sync_*`) . . . 15
Service control (`nano_*`, `wo_*`, `basicmodel_*`) . . . 16
Other targets . . . 16
Key overridable variables . . . 16
Server and Client Setup . . . 17
Architecture . . . 17
Key files . . . 17
Quickstart . . . 17
Configuration . . . 17
Session portability . . . 18
Security notes . . . 18

Truth **18**
Plural Truth and Shared Empathy in the Design of WikiOracle . . . 18
The Problem With Current Model Development . . . 18
A Plural Model of Truth . . . 19
Empathy as Shared Constraint . . . 19
Relation to the Wikipedia Model . . . 19
Stability . . . 20
Distributed Truth vs Consensus . . . 20

Ethics **20**
Truth Symmetry . . . 20
Architectural ethics vs policy ethics . . . 21
Symmetry as a foundation of ethical reasoning . . . 21
Reciprocity symmetry . . . 21
Reversibility symmetry . . . 22
Harm symmetry . . . 22
Universality (Golden Rule) – architectural enforcement . . . 22
Truth improves ethics . . . 23
Ethical knowledge geometry . . . 23
Architectural ethical primitives in WikiOracle . . . 23
Symmetry operators . . . 23
Epistemic humility . . . 24
Distributed governance . . . 24
The identity crisis and surveillance capitalism . . . 24

Privacy and Security **24**
Privacy . . . 24
Why news facts are session-only . . . 24
Personal Identity . . . 25
Entanglement Policy . . . 25
Entanglement avoidance and sovereignty . . . 26
Reasoning about generalities vs specifics . . . 26
Three Channels . . . 26
Why particular facts always train weights . . . 27
Spatiotemporal Extent . . . 27
Resulting Architecture . . . 28
Zero-Knowledge and Selective Disclosure . . . 28
Errata . . . 28
Security . . . 29
Private Data . . . 29

API Keys . . . 29
Identity . . . 29
3Reverse Proxy . . . 30
Cross-Site Scripting (XSS) . . . 30
CORS . . . 30
File System . . . 31
Third-party Scraping . . . 31

Freedom **31**
Entanglement Policy . . . 31
Entanglement Policy . . . 31
Why particular facts always train weights . . . 32
Global truth with local freedom (LFGC) . . . 32
Entanglement avoidance and sovereignty . . . 32
Conceptual knowledge . . . 33
Interpretive reasoning . . . 33
Personal experience . . . 33
Reasoning about generalities vs specifics . . . 33
Three Channels . . . 33
Universal Knowledge (Database) . . . 33
Particular Knowledge (Training Input) . . . 33
Feelings . . . 34
Spatiotemporal Extent . . . 34
Resulting Architecture . . . 35
Zero-Knowledge and Selective Disclosure . . . 35
Notes . . . 35

Voting **36**
Call sequence . . . 36
Topology . . . 36
Per-beta conversation control . . . 36
Cycle prevention . . . 36
All output is truth . . . 37

Logic **38**
Feelings – outside the truth lattice . . . 38
Fact kinds – knowledge (universale) vs news (particulars) . . . 38
Logical operators in WikiOracle's truth system . . . 38
Fuzzy Kleene Logic with Feeling . . . 38
Strong Kleene evaluation . . . 39
Non: non-affirming negation . . . 39
Precedent in Buddhist logic . . . 40
Why $1 - 2|a|$ is the right formula . . . 40
Fuzzy logic interpretation . . . 41
Relation to the Madhyamaka problem . . . 41
Expressive necessity: why {and, or, not} is incomplete without non . . . 41
The regularity barrier . . . 41
What non adds: the detection functions . . . 42
The completeness result . . . 42
A Two-Axis Epistemic Algebra . . . 42
See also . . . 43

Training **43**
DegreeOfTruth (DoT) . . . 43
Dynamic Systems Perspective . . . 43

Zero-Mean Balance . . . 44
Sigmoid Warmup as Annealing Schedule . . . 44
Sensation – Preprocessing Pipeline . . . 44
JSONL Record Types . . . 44
Korzybski IS Detection . . . 45
Usage . . . 45
Online Training . . . 45
Purpose . . . 45
User Identity . . . 46
Pipeline . . . 46
Device Configuration . . . 47
Training Algorithm – DoT-Annealed Selective Training . . . 47
SFT Corpus Preparation . . . 50
Server TruthSet . . . 50
Anti-Capture . . . 51
Dissonance Detection and Pluralistic Truth (TODO) . . . 51
OpenClaw Integration . . . 52
Server TruthSet Visibility (Debug Mode) . . . 52

Implementation **53**
System overview . . . 53
Components . . . 53
Server endpoints . . . 54
Data flow: client <-> server . . . 54
Chat routing . . . 54
Thought-Free Mode . . . 54
Reducing Inferential Truth to Facts and Feelings . . . 55
State library (`bin/state.py`) . . . 55

Config **56**
Server . . . 56
Truthset . . . 56
Evaluation . . . 57
Training . . . 57
Allowed URLs . . . 58
Providers . . . 58
Built-in providers . . . 59
API key precedence . . . 59
Runtime precedence . . . 60
Environment variables . . . 60
Provider-specific environment overrides . . . 60
OpenClaw Plugin Config . . . 61
CLI flags . . . 61
Schema . . . 62

State **62**
Structure . . . 62
Default state format . . . 62
Server State . . . 62
Security considerations . . . 63
Client . . . 63
UI preferences . . . 63
Conversations . . . 64
Grammar . . . 64

Conversation attributes . . . 64
Message attributes . . . 64
Example . . . 64
Tree operations . . . 65
TruthSet . . . 65
Shared truth attributes . . . 65
Truth kinds . . . 65
Example . . . 65
Static vs dynamic truth . . . 66
Serialization . . . 66
Merge . . . 66
In-memory representation . . . 66
Schema . . . 67
Data model . . . 67
On disk – XML . . . 67
In memory – nested tree . . . 68
Grammar . . . 68

User Interface **69**
Rendering . . . 69
Context for the upstream LLM . . . 69
Tree visualisation (D3) . . . 69
Chat view . . . 69
Rendering pipeline (end to end) . . . 69
Interactions . . . 70
Tree panel . . . 70
Keyboard navigation . . . 70
Chat panel . . . 70
Merge semantics . . . 70
Branching . . . 71
Settings Dialog . . . 71
Settings Controls . . . 71
Settings Persistence . . . 71
Truth Weight Slider . . . 71
Max Truth Entries . . . 72
Store Concrete Facts . . . 72
Legacy Migration . . . 72
Server TruthSet Display . . . 72
User Interface Strings . . . 72
Truth editor . . . 72
Dropdown labels . . . 72
Descriptions . . . 72
Templates . . . 73
Empty state . . . 73

Proposed License **73**
WikiOracle Licensing Architecture . . . 73
Four-Layer Conceptual Model . . . 73
License Comparison Table . . . 74
GPL-3.0 . . . 74
CC BY-SA 4.0 . . . 74
Apache-2.0 . . . 74
Rationale for Layered Licensing . . . 74
Software (GPL-3.0) . . . 74

Knowledge Content (CC BY-SA) . . . 74
Structured Data (CC0) . . . 74
Model Weights (Apache-2.0) . . . 74
Training Data Provenance Policy . . . 75
AI-Generated Content Policy . . . 75
Summary . . . 75

WikiOracle

Revision: 2026.02.27

An open-source architecture for truthful AI.

WikiOracle is a truthful, explainable LLM system designed as a public good – the Wikipedia model applied to artificial intelligence.

The Problem

For-profit corporations are using our data – sourced from billions of people – to train models that are teaching our children. Those models hallucinate. They can't explain themselves. They are vulnerable to ideological capture and data-driven manipulation, especially under online learning. And the knowledge they encode is locked behind proprietary walls.

Most large AI systems today are built around a single global objective function, centralized data aggregation, hidden alignment rules, and implicit averaging over moral and cultural differences. The result is predictable: minority viewpoints are quietly averaged away, the loudest groups shape the model at scale, a single model becomes an authority node that everyone depends on, and predictive advantage converts into economic or political dominance.

When this happens, wisdom stops being a shared good and becomes a strategic asset.

What Makes WikiOracle Different

Truth

WikiOracle does not optimize for fluency and bolt on truthfulness as an afterthought. Truthfulness is the primary design constraint. Every claim traces back to explicit trust entries carrying certainty values on [-1, +1]. Reasoning chains and citations are inspectable. Grounded models are less prone to hallucination and capture, and claims can be contested, improved, or revised openly.

Data Soverignty

WikiOracle is local-first. Your conversation state, your trust entries, and your configuration live on your machine – not on a corporate server accumulating hidden central memory. You can export, merge, and port your sessions freely. Your data is yours.

The remote server is strictly stateless: it holds no per-user conversation, no trust table, and no session memory between requests. Every call arrives with the state it needs and leaves nothing behind.

For off-machine backup and sharing, WikiOracle can optionally stage your `state.xml` and `config.xml` to your own Dropbox as AES-256 encrypted ZIP archives (via `pyzipper`). The encryption password is held only by you – Dropbox sees ciphertext, and the server never persists either the password or the decrypted contents.

Sharing Truth via QR Code

You can share a conversation or a truth set by publishing its encrypted state bundle to Dropbox and exposing the Dropbox link plus the decryption key inside an `<authority>` XML element. WikiOracle renders that

element as a QR code in the client – scan it from another device to copy the authority URI, paste it into the recipient's Truth editor, and their client will fetch and decrypt the bundle, importing your conversation and trust entries at whatever trust weight they choose to grant it.

Democracy

No single actor – company, state, foundation, or maintainer group – can silently become the epistemic root for everyone else. WikiOracle supports multiple points of view, each with its own trust map and standards of evidence. Where serious disagreement exists among credible sources, the system represents the dispute rather than smoothing it away. Minority viewpoints are preserved, not averaged into oblivion. And if governance ever fails these obligations, forking is a constitutional right.

Distribution

Instead of one model that claims to know everything, WikiOracle builds a network of trust. Authorities are pointers to external knowledge bases whose entries are imported with scaled certainty – we trust what they trust, to a degree. You choose who to trust and how much. Multiple LLM providers serve as "other minds" whose outputs become evidence, not unquestionable authority. Trust is transitive but attenuated, distributed but structured. A distributed truth network prevents appropriation. Open truth does disrupt business models that depend on information asymmetry, extractive IP capture, and strategic opacity, but that disruption is corrective.

Current Prototype

The initial prototype is intentionally modest and low-cost:

- Hierarchical, multi-LLM architecture for runtime-configurable Hierarchical Mixture of Experts.
- User-specified truth sets (consisting of facts, feelings, references, operators, authorities, and providers)
- Online learning constrained by trust and epistemic grounding
- Extends NanoChat with Truth Sets using Retrieval-Augmented Generation
- Allows feasible experiments in LLM architectures on rented compute (~$100 scale)

Longer-Term Direction

If WikiOracle proves viable at small scale, the architecture can be evaluated and extended to larger open models. The broader aim of WikiOracle is to explore whether architectural commitments to truth can enable honest self-explanation, reduce the need for ad-hoc guardrails, and support AI systems that function as durable public goods. See FutureWork.md

How to Contribute

Contributions of many kinds are welcome:

- ML research and implementation
- xAI, interpretability, and safety analysis
- Epistemology, philosophy of science, and governance critique
- Documentation, evaluation, and testing

Getting Started

See the Installation Guide for build, deploy, and runtime instructions.

Quickstart:

```
pip install -r requirements.txt
python bin/wikioracle.py
```

Documentation

Server and client documentation lives in `./doc`:

File	Topic
Config.md	Configuration format, settings reference, and environment variables
Constitution.md	Non-negotiable invariants for WikiOracle truth and governance
Ethics.md	Ethical AI through truth architecture, entanglement policy, and truth development
Freedom.md	Freedom, entanglement policy, and worldline-capture constraints
FutureWork.md	Server roadmap and future directions
Voting.md	Hierarchical Mixture of Experts architecture and voting model
Implementation.md	Implementation notes
Installation.md	Build, deploy, and runtime instructions
Logic.md	Logical operators, Strong Kleene evaluation, and derived truth
PrivacyAndSecurity.md	Privacy and security considerations
ProposedLicense.md	Proposed licensing architecture
Socrates.pdf	PDF reference document
State.md	State file format, conversation tree, truth table, and serialization
Training.md	Training pipeline, DegreeOfTruth, and NanoChat integration
Truth.md	Plural truth, POVs, empathy, and certainty semantics
UserInterface.md	Canonical client UI strings and labels
WikiOracle.md	WikiOracle design overview
WikiOracle.pdf	PDF version of the WikiOracle overview

BasicModel AI documentation lives in `./basicmodel/doc`:

File	Topic
Architecture.md	Six-space bidirectional pipeline, invertible layers, LDU
BasicModel.md	Four-space cognitive model theory, symmetric perception
BuddhistParallels.md	Buddhist epistemology parallels, pramana theory, tetralemma
Ergodic.md	Ergodic exploration, gradient energy sensor, simulated annealing
Grammar.md	Formal grammar mapped to neural architecture spaces
Language.md	SyntacticLayer, CNF grammar rules, method implementations, thought-free mode
Logic.md	Subsymbolic/symbolic operators, truth fields, luminosity
MachineMinds.md	Weight ergodicity, network invertibility, output certainty
Params.md	XML configuration schema for all model hyperparameters
Reasoning.md	Reasoning methods, TruthLoss, partitioned symbolic space, contemplative awareness
Spaces.md	InputSpace through OutputSpace specifications
Training.md	Two-phase training: embedding pretraining + network training

Research Materials

Supporting papers live in `doc/research/`:

File	Type
1711.00937v2.pdf	arXiv paper PDF
2309.11495v2.pdf	arXiv paper PDF
2311.05232v2.pdf	arXiv paper PDF
2312.10997v5.pdf	arXiv paper PDF
2403.05156.pdf	arXiv paper PDF
2409.18786v1.pdf	arXiv paper PDF

File	Type
2411.06528v2.pdf	arXiv paper PDF
2412.12472v2.pdf	arXiv paper PDF
2503.22759v1.pdf	arXiv paper PDF
2510.06265v2.pdf	arXiv paper PDF
2511.03529v1.pdf	arXiv paper PDF
2601.11199v1.pdf	arXiv paper PDF
BF_ICDCS_2022.pdf	conference paper PDF

WikiOracle

Revision: 2026.02.27

Architectural Overview

The system is governed by two file types:

- **State files** (`.xml`): contain client identity, UI preferences, conversations, and truth entries (validated by `data/state.xsd`).
- **Config files** (`.xml`): contain provider defaults and credentials, server policy, and runtime defaults (validated by `data/config.xsd`).

Client

The state of WikiOracle is maintained by the client. It has the following information:

- **Client**: has information about the user and the file itself
  - **Client Name**: display name shown for user-role messages
  - **Client ID**: stable client identifier
  - **UI**: browser-side default interface settings
    - **Layout**: horizontal or vertical layout, with legacy `flat` still supported
    - **Theme**: system, light, or dark
    - **Model**: currently selected model override
    - **Confirm Actions**: whether deletes and merges require confirmation
    - **Splitter**: saved tree/chat panel split position
- **Conversation**: consists of queries, responses, and sub-conversations arranged as a tree
  - **Query**: text from the user
  - **Response**: text from the selected main provider
  - **Selection**: the currently selected conversation path
  - **Branching**: child conversations can diverge from any earlier point
- **TruthSet**: a set of beliefs, each of which has a Degree of Truth
  - **Direct Truth**: things that we know directly
    - **Feeling**: statements which are beyond true or not true, like poetry or opinion
    - **Fact**: concrete and abstract propositions about the world and language itself
    - **Operator**: logical operators such as And, Or, Not, and Non
  - **Indirect Truth**: things that we know only indirectly
    - **Reference**: a URL or citation that grounds a claim
    - **Provider**: another oracle that can provide truth and optionally participate in the conversation
    - **Authority**: a reference to another set of conversations and truths

Server

The server executes the client state, based on a configuration file and a corpus of truth that it maintains which is identical to the TruthSet of a state file.

The configuration file on the server has information such as the following:

- **Server**: runtime and training parameters
  - **Server Name**: human-readable server name
  - **Server ID**: stable server identifier
  - **Stateless**: whether the server is allowed to write to disk
  - **URL Prefix**: optional route prefix for reverse-proxy deployments
  - **TruthSet**: parameters governing the server's TruthSet
    - **Truth Symmetry**: asymmetric-harm checking under identity exchange
    - **Store Concrete**: whether spatiotemporally-bound facts persist
    - **Truth Weight**: how much the TruthSet affects RAG and training
  - **Evaluation**:
    - **Temperature**: sampling temperature
    - **Max Tokens**: maximum response length
    - **Timeout**: request timeout for provider calls
    - **URL Fetch**: whether URL fetching is allowed during evaluation
  - **Training**: continuous learning subsystem
    - **Enabled**: master switch for post-response learning
    - **Truth Corpus Path**: path to the server TruthSet
    - **Truth Max Entries**: max server truth entries before trimming
    - **Learning Rates**: `alpha_base`, `alpha_min`, `alpha_max`
    - **Merge Rate**: moving-average merge speed for truth updates
    - **Device**: `cpu`, `cuda`, or `auto`
    - **Dissonance**: contradiction detection and penalty
    - **Warmup / Clipping / Anchoring**: `warmup_steps`, `grad_clip`, `anchor_decay`
  - **Allowed URLs**: whitelist for authority and provider fetches
- **Providers**: one or more upstream LLM provider definitions
  - **Default Provider**: provider selected on startup when the request does not override it
  - **Shared Prompt Fields**: `context`, `output`, `truth_context`, and `conversation_context`
  - **Provider**: a single provider entry keyed by name
    - **Display Name**: label shown in assistant messages
    - **Username**: account login or email
    - **URL**: API endpoint
    - **API Key**: authentication credential
    - **Default Model**: model used when none is selected explicitly
    - **Timeout / Streaming**: per-provider request behavior
  - **Built-in Providers**: WikiOracle, OpenAI, Anthropic, Gemini, Grok, and OpenRouter

Conversation

A conversation happens first by processing the Indirect Truth entries of the TruthSet: single references are fetched, the truth of authorities are collected, and providers (which are other minds) are asked for their input to the current Query. The result of that is a TruthSet that consists only of Direct Truths.

All truths (except Feelings) have a Degree of Trust in the range [-1, 1], where -1 is fully untrusted, 0 is unknown, and 1 is fully trusted. Feelings are always trusted, but do not count as evidence when deliberating. So computation over these truths looks like a network of trust that involves various sources and our own intuitions (whihc count, even though they cant provide evidence).

Each of those facts is then compared to the TruthSet stored by the server.

TruthSet

- **Trust entries** carry certainty values in [-1, +1] using Kleene ternary/fuzzy logic – from certainly true (+1) through ignorance (0) to certainly false (-1).
- **Logical operators** (and/or/not/non under Strong Kleene semantics) compute derived certainty over the TruthSet.
- **Authorities** reference external knowledge bases, enabling transitive trust with certainty scaling.
- **Providers** are external LLMs used as expert consultants whose responses become sources with associated certainty.
- **Feelings** are subjective statements (opinions, poetry, hedged claims) occupying the "neither" position in the tetralemma. They influence evaluation but are excluded from training and TruthSets.
- **References** are external source citations (Wikipedia, Snopes, etc.) that ground claims in verifiable sources, participating in the TruthSet alongside facts.

The UI-selected provider acts as the "mastermind," synthesizing all evidence – facts, references, operator-derived certainty, authority imports, and provider consultations – into a final response.

Example

As a somewhat fun example, we consider how WikiOracle can be used to create a voting system that operates in real-time as a Hierarchical Mixture of Experts composed of multiple LLMS (or even a single LLM with mutliple truth sets).

The "alpha" conducts the vote. So there is a conversation in which the user asks a question. The state file of the alpha contains, in addition to various facts, feelings, and other sources of truth, two providers which we will call the "betas". When the user directs a query to the alpha, the alpha first turns its indirect truth into direct truth: that means evlauating the providers. So, the Query from the user is passed to the Betas.

Each of the Betas sees the query and is asked to respond with a Response to the Query, and optionally to provide its Facts and Feelings that are relevant to that Query. In a sense, they are voting on that query, and providing their own reasons for having done so.

Finally, the Alpha see the query and the Responses (or votes) cast by the betas. It also sees the facts and feelings that they have returned, which are incorporated into the TruthSet. So it has this mixture of its own truth and truth provided by each of the Betas, and based on their advice and its trust of each Beta, it concludes with a Respons of its own. This construct is a bit non-traditional in terms of a linear conversation, and it fact it creates a diamond pattern in the tree view of the conversation.

For more information, see Voting.

WikiOracle Constitution

Revision: 2026.02.27

Preamble

WikiOracle is an open-source architecture for truthfulness: a way to enlist the power of LLMs while preserving truth in the face of centralized, opaque "global corporate mental models" (including those embedded in proprietary systems).

WikiOracle is the Wikipedia model applied to LLMs:

- anyone can contribute claims, counterclaims, evaluations, and tests
- revision is expected and legible
- provenance and dispute are first-class, not edge-cases

However, it is also distributed, possibly encrypted, and hopefully kind:

- Truth does not need to be centralized to be shared.

- An unempathetic body of truth can be prevented by refusing selfish truths (truth must be universally true).

Truthfulness is not a one-time achievement. It is an ongoing, open engineering and governance effort.

Even if the reasoning machine is large and unified, the data and trust it operates on must remain distributed and under individual control. Epistemic contributions are people-owned and revocable by the data providers. Answers must be conditioned on what each user chooses to trust and believe to be true, not forced into a single impoverished consensus view of reality.

This constitution defines the non-negotiable invariants for WikiOracle's truth system and for changes to it. Implementation details and deeper theory live in the rest of the `doc/` directory.

Core Commitments

WikiOracle is architected not merely as a system of prediction, but as a shared epistemic commons. It must:

- Increase collective agency without concentrating domination.
- Represent the concerns of all creatures, not just the operator.
- Preserve truth as a shared human resource.

Therefore:

1. **Truthfulness over fluency** Prefer grounded, falsifiable, uncertain, or incomplete answers over smooth, unsupported ones.
2. **Truth is not consensus** A single, averaged narrative is not the goal. The system must preserve real disagreement where it exists.
3. **Plural points of view** The system must support multiple Points of View (POVs), each with its own trust map and standards of evidence.
4. **Transparency by default** Claims, confidence, provenance, and update rationales must be inspectable and reproducible.
5. **Reversibility and accountability** High-impact changes must be attributable, testable, and reversible.
6. **Public benefit and anti-capture** No single actor (company, state, foundation, or maintainer group) should be able to silently become the epistemic root for everyone else.

Truth Primitives (What the System Is Allowed to Believe)

WikiOracle's truth layer is composed of explicit, user-visible primitives stored in state, all of which have some Degree of Trust (except for Feelings):

1. **Feelings**: Feelings are non-falsifiable (direct) perceptions of reality.
2. **Facts**: atomic propositions, citations, or evidence. They carry a `certainty` in `[-1..0..+1]` (Fuzzy Kleene: believed, unknown, disbelieved).
3. **References**: pointers to external object (URLs).
4. **Operators**: explicit computed relationships: AND, OR, NOT, NON.
5. **Authorities**: pointers to external knowledge bases (remote `state.xml` files, ORCIDs, or DIDs): we trust what they trust, to some degree.
6. **Providers**: external AIs used as tools ("other minds") whose outputs become evidence, not unquestionable authority.

All truth computations must remain legible as operations over these primitives. If the system "knows" something, it should be possible to point to what it is grounded in; otherwise it is merely intuition. See Truth.md

Plurality, Dispute, and Minority Preservation

1. **POV-conditioned conclusions** When different POVs trust different sources, the system should be able to present conclusions conditioned on the selected POV/trust map.
2. **Overlaps are valuable** When independent POVs converge on the same claim, that agreement should be surfaced explicitly as a robustness signal.
3. **Disputes stay visible** Where serious disagreement exists among credible sources or POVs, the system must represent the dispute rather than smoothing it away.
4. **Minority protection** Evidence-supported minority viewpoints must not be excluded solely by majority preference, institutional pressure, or convenience.

Independence From Any Single AI Vendor

WikiOracle may use proprietary or open models as providers, but:

1. **No provider is privileged** Data is revocable and belongs to the client.
2. **Providers are evidence generators** Provider outputs should be treated as non-authoritative contributions with a trust value like any other entry.
3. **Replaceability is required** The system must remain operable if any single provider becomes unavailable, hostile, or compromised.

Authority Delegation Must Be Bounded and Secure

Authority entries exist to enable decentralized truth (a network of trust) without collapsing into a single global oracle.

1. **Trust decreases with every hop** Importing an authority must not recursively fetch authorities of authorities.
2. **Certainty scaling** Imported claims must be scaled by the authority's certainty (trust is transitive but attenuated).
3. **Namespaced IDs** Imported entries must be namespaced to prevent collisions and to preserve provenance.
4. **Explicit source selection** Users (or POV definitions) must explicitly choose which authorities to trust; there is no implicit global root.
5. **Operational safety** Fetching and caching authorities must be rate-limited and size-limited, and restricted to safe URL schemes.

Local-First Data and Auditability

1. **Client-owned Particular State** The default posture is local-first: user conversation state and news lives on the user's machine and are portable.
2. **Server-owned Universal State** A shared hosted service accumulates only anonymized knowledge in a TruthSet which serves as a trusted source of facts.

See PrivacyAndSecurity.md

Safety as Freedom, Empathy, and Truth

WikiOracle's truthfulness effort must not trade away human welfare or agency:

1. **Freedom:** increase distributed agency, not centralized leverage. AI must not be used to transgress the freedom of others. Data sovereignty and freedom to use the platform are non-negotiable.
2. **Empathy:** represent the concerns of all creatures, not just the operator. Preserve dignity, minimize harm, and make uncertainty explicit. De-emphasize egocentric optimization that externalizes costs.
3. **Truth:** keep truth auditable and non-proprietary; do not convert epistemic advantage into coercive control. The truth is not a commodity: data formats, tests, and willingly shared truth must be open and reproducible. Make identification and surveillance impossible.

See Freedom.md, Ethics.md, and Truth.md.

Installation

Environment and System Preparation

Requirements

- Python 3 and `make`.
- `uv` for NanoChat environment setup (`make install` will bootstrap it if missing).
- AWS CLI configured for build-host launch workflows (`make train HOST=build` and related targets).
- SSH keys:
  - EC2 training key (`~/.ssh/nanochat-key.pem`, auto-created by remote tooling as needed).
  - Lightsail key (`./wikiOracle.pem` by default, configurable via `WO_KEY_FILE`).

Python dependencies

`make install` creates both virtual environments and installs all dependencies:

```
make install              # CPU/MPS (default)
make install ARCH=gpu     # GPU/CUDA
```

This sets up:

- `.venv` – the WikiOracle shim server (`flask`, `requests`, etc. from `requirements.txt`)
- `nanochat/.venv` – the NanoChat local LLM (managed by `uv`)

Data and tokenizer bootstrap

```
make build_data
make build_tokenizer
```

Environment variables

Variable	Default	Purpose
`WIKIORACLE_STATE_FILE`	`state.xml`	Path to local state file (WikiOracle State XML)
`WIKIORACLE_BASE_URL`	`http://127.0.0.1:8000`	Upstream NanoChat-compatible base URL
`WIKIORACLE_API_PATH`	`/chat/completions`	Upstream chat path
`WIKIORACLE_STATELESS`	(unset)	Set truthy to disable all writes and use in-memory state
`WIKIORACLE_URL_PREFIX`	(unset)	Optional reverse-proxy path prefix
`WIKIORACLE_BIND_HOST`	`127.0.0.1`	Network interface to bind
`WIKIORACLE_BIND_PORT`	`8888`	Server port

The Makefile: Running and Building

All local and remote workflows are orchestrated through the Makefile.

Top-level targets

These are the primary entry points. For a local WikiOracle + NanoChat stack, use `install`, then the `nano_*` and `wo_*` service targets. `build` is still present, but it delegates to `train`, which currently runs the BasicModel training pipeline.

Target	Purpose
`make install`	Create `.venv` (shim + NanoChat), install all deps
`make build`	Alias for `make train` (currently the BasicModel pipeline)
`make nano_train`	Full NanoChat training pipeline
`make sync HOST=local`	Sync app to ArborMini
`make sync HOST=remote`	Sync app + checkpoints to/from production server
`make sync HOST=build`	Deploy active remote training artifacts to WikiOracle
`make run HOST=local`	Start WikiOracle Flask shim locally (foreground)
`make up HOST=remote`	Restart both services on production
`make down HOST=remote`	Stop both services
`make all`	Full pipeline: install + train + eval + report

Training (`train_*`)

Target	Purpose
`make train HOST=local`	BasicModel training pipeline
`make train HOST=build`	Launch remote GPU instance, copy repo, start training
`make nano_train`	Full NanoChat pipeline: data + tokenizer + SFT model
`make train_pretrain`	Pretrain base model (`ARCH=cpu|gpu`)
`make train_finetune`	Supervised fine-tuning (`ARCH=cpu|gpu`)
`make train_deploy HOST=build`	Launch remote instance, train, then deploy to WikiOracle
`make train_retrieve HOST=build`	Pull artifacts from build instance, terminate it
`make train_ssh HOST=build`	SSH into running build instance
`make train_status HOST=build`	Check build instance state
`make train_logs HOST=build`	Tail training log on build instance

Test / Evaluation (`test_*`)

Target	Purpose
`make test HOST=local`	Run WikiOracle + BasicModel tests
`make test_all HOST=local`	Run WikiOracle + BasicModel `test_all` target
`make test_unit HOST=local`	Run unit tests
`make test_basicmodel HOST=local`	Run BasicModel tests
`make test_eval HOST=local`	Evaluate model (`ARCH=cpu|gpu`)

Run / Inference (`run_*`)

Target	Purpose
`make run HOST=local`	Start WikiOracle local shim (foreground)
`make run_debug HOST=local`	Start WikiOracle local shim (debug mode)
`make run_init HOST=local`	Remove state files for a fresh start
`make run_cli HOST=local`	Chat with NanoChat directly (CLI)
`make run_web HOST=local`	Run the NanoChat web extension

Sync (`sync_*`)

Target	Purpose
`make sync HOST=local`	Sync app to ArborMini
`make sync HOST=remote`	Sync app + checkpoints to/from production server
`make sync HOST=build`	Deploy active remote training artifacts to WikiOracle
`make sync_checkpoint_pull`	Pull fine-tuning weights from production
`make sync_checkpoint_push`	Push fine-tuning weights to production

Service control (`nano_*`, `wo_*`, `basicmodel_*`)

NanoChat and WikiOracle support local (PID file + background process) and remote (`systemctl`) operation, controlled by `HOST=local|remote`. The `sync` target additionally accepts `HOST=build` to deploy artifacts from the active remote training instance. Training uses `HOST=local|build`, while run/test targets are local-only and reject non-local hosts. BasicModel is registered but does not yet have an inference server.

Target	Purpose
`make nano_start` / `nano_stop` / `nano_restart`	Manage local or remote NanoChat
`make nano_status` / `nano_logs`	Check NanoChat PID/service state
`make wo_start` / `wo_stop` / `wo_restart`	Manage local or remote WikiOracle
`make wo_status` / `wo_logs`	Check WikiOracle PID/service state
`make basicmodel_status`	Check BasicModel status (stub)

For local debugging, the typical workflow is:

```
make nano_restart NANO_MODEL_TAG=d26 NANO_DEVICE_TYPE=cpu NANO_DTYPE=float32
make wo_restart
make nano_status wo_status
```

This starts an unchanged local NanoChat backend on port `8000` and the WikiOracle shim on port `8888`. `NANO_MODEL_TAG` selects the checkpoint, `NANO_DEVICE_TYPE` selects `cpu` or `cuda`, and `NANO_DTYPE` controls the runtime dtype.

Other targets

Target	Purpose
`make openclaw_setup/run/test`	OpenClaw extension management
`make doc_pdf`	Generate PDF from all `doc/*.md`
`make doc_report`	Generate training report
`make clean` / `clean_all`	Remove caches (and optionally venvs)

Key overridable variables

Variable	Default	Purpose
`ARCH`	`cpu`	Target architecture (`cpu` or `gpu`)
`HOST`	`local`	Target host for `up/down/nano_*/wo_*`
`NANO_MODEL_TAG`	`d26`	NanoChat checkpoint tag used by `nano_start` / `nano_restart`
`NANO_SOURCE`	`sft`	NanoChat input family passed to `scripts.chat_web`
`NANO_DTYPE`	`float32`	NanoChat inference dtype
`NANO_DEVICE_TYPE`	`cpu`	NanoChat device type (`cpu` or `cuda`)
`NANO_PORT`	`8000`	NanoChat server port
`WO_BIND_HOST`	`127.0.0.1`	WikiOracle bind host

Variable	Default	Purpose
`WO_PORT`	`8888`	WikiOracle server port
`NPROC`	`1`	GPUs per node for torchrun

Server and Client Setup

Architecture

WikiOracle runs as a local Flask server (`bin/wikioracle.py`) that proxies chat requests to any LLM provider (NanoChat, OpenAI, Anthropic, Gemini, Grok, OpenRouter) while keeping all conversation state on the local filesystem. The remote server at `wikiOracle.org` operates in stateless mode.

Key files

File	Role
`bin/wikioracle.py`	Flask server (binds to `127.0.0.1:8888` with self-signed TLS). Proxies chat upstream, persists state
`bin/config.py`	Config loading, XML I/O, provider registry, schema-driven normalization. Auto-generates `server_i`
`bin/state.py`	State validation, XML I/O, collision-safe merge, context-delta extraction
`bin/response.py`	Chat pipeline, provider coordination, voting fan-out, online training pipeline
`bin/truth.py`	Trust processing, authority resolution, operator engine (and/or/not), DegreeOfTruth, PII detection
`config.xml`	Server configuration: provider credentials, chat settings, UI defaults, server identity. Validated by `da`
`state.xml`	Client-owned state: header (user identity, timestamps), conversations, truth entries. Validated by `da`
`client/index.html`	Single-page web UI shell with chat, settings, and merge tools
`test/test_*.py`	Automated tests for state, stateless contract, prompt bundles, authority, derived truth, voting, onlin

Quickstart

```
make install                    # create venvs, install all deps
make nano_restart NANO_MODEL_TAG=d26 NANO_DEVICE_TYPE=cpu NANO_DTYPE=float32
make wo_restart
make nano_status wo_status
```

Open `https://127.0.0.1:8888` in a browser and accept the self-signed certificate.

For a short CLI smoke test against the local stack:

```
./.venv/bin/python ./bin/wo -k --provider wikioracle --model nanochat "Yes or no only?"
```

If you only want the WikiOracle shim in the foreground, use:

```
make run HOST=local
```

Or manually:

```
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip setuptools wheel
pip install -r requirements.txt
python3 bin/wikioracle.py
```

Configuration

The server reads `config.xml` at startup. On first run, it auto-generates a `server_id` (UUID4) and writes it back. Provider API keys can be set in the XML or via environment variables (`ANTHROPIC_API_KEY`, `OPENAI_API_KEY`, `GEMINI_API_KEY`, `XAI_API_KEY`).

User identity (name, user_id) lives in the state file, not in the config – the client is the authority for state, and the server is the authority for config.

Session portability

Export conversations from phone or browser as `llm_YYYY.MM.DD.HHMM.xml`, then merge into a local project's `state.xml`:

```
python3 bin/wikioracle.py merge llm_*.xml
```

This provides a clean integration path with Claude Code, OpenAI Codex, or any local tooling that can read the state file for project context.

Security notes

- WikiOracle credentials are not copied onto EC2 training instances.
- Deployment excludes local/dev artifacts from rsync (`.venv`, caches, local data, `.env`, etc.).
- The local server binds to `127.0.0.1` by default – not exposed to the network.
- TLS is enabled by default with a self-signed certificate (use `--no-ssl` to disable).

Truth

Thoughts are non-rivalrous: if someone repeats your idea, you still possess it. Therefore an idea as such cannot be a property object. ~ Immanuel Kant

Plural Truth and Shared Empathy in the Design of WikiOracle

Truth is not the same thing as consensus.

- Different communities trust different sources.
- Different disciplines apply different standards of proof.
- Different cultures interpret the same facts through different lenses.

A healthy knowledge system does not erase these differences. It preserves them, makes them visible, and shows where they overlap. WikiOracle should not aim to speak with a single authoritative voice. It aims to:

- Show what holds across many points of view.
- Show where serious disagreements remain.
- Explain why disagreement exists.
- Make uncertainty explicit.

Truth is strongest when its mechanism of trust is transparent, not when it is uniform.

The Problem With Current Model Development

Most large AI systems today are built around:

- A single objective function that optimizes over a shared global set of truth.
- Centralized data aggregation.
- Implicit averaging over moral and cultural differences.
- Hidden alignment rules.
- Proprietary control of core knowledge.

This produces predictable risks:

Collapse into Consensus - Minority viewpoints are quietly averaged away.
Preference Capture - The loudest or most powerful groups shape the model at scale.
Epistemic Centralization - A single model becomes an authority node, increasing dependence.

Conversion of Knowledge into Leverage - Predictive advantage becomes economic or political dominance.

When these risks manifest, wisdom stops being a shared good and becomes a strategic asset.

A Plural Model of Truth

WikiOracle should maintain multiple Points of View (POVs), each with its own trust map.

Each POV may:

- Trust different sources.
- Weigh evidence differently.
- Interpret claims differently.

Instead of collapsing these into one answer, the system should:

- Present POV-conditioned conclusions.
- Identify robust overlaps across perspectives.
- Preserve live disputes.
- Keep minority views visible and searchable.

Distrust between communities is not a defect. It creates epistemic distance, preventing forced consensus. A plural system can remain coherent without being uniform. This model of truth is simultaneously multicultural (respecting diverse cultural frameworks), multivalent (allowing claims to carry different weights under different perspectives), and pluralistic (treating the coexistence of competing truths as a structural feature rather than a flaw).

Empathy as Shared Constraint

Plural truth alone is not sufficient. Empathy needs to be architectural. It means enforcing how disagreement is handled:

- When Truth conflicts, meaning that the same Truth is held to be both True and False, each truth should be contextualized in its own reference frame.
- Feelings cannot conflict, because they are authoritative from different points of view.

As a first take at encoding empathy, a Golden Rule is encoded by allowing the choice of symmetric truths. An unempathetic body of truth can be prevented by refusing selfish truths (truth must be universally true).

Relation to the Wikipedia Model

Wikipedia provides an important precedent:

- Open participation (achieved through low barrier to entry or free use without advertising).
- Transparent citation (achieved through a corpus of truth within which you can decide what you trust and distrust).
- Community moderation (achieved by reaching agreement within a single reference frame or creating separate reference frames).
- Distributed subject-matter authority (achieved by creating separate reference frames or trusted authorities)

WikiOracle extends the strengths of Wikipedia while going further:

- Support structured parallel POVs instead of a single narrative by allowing separate domains of trust.
- These multiple trust domains are visible rather than implicit.
- Model disagreement explicitly rather than smoothing it away.

Stability

Growth in wisdom is not destabilizing when:

- Knowledge remains open and auditable.
- No single actor can monopolize epistemic advantage.
- Minority perspectives are preserved.

Such a system may disrupt business models that depend on opacity or information asymmetry. That disruption is corrective, not destructive.

Distributed Truth vs Consensus

WikiOracle attempts to achieve a distributed truth, and shares much of the design philosophy of Wikipedia:

- **Radical Accessibility** – Wikipedia is freely accessible worldwide and allows anyone to both read and contribute without institutional gatekeeping.
- **Citation Norms** – Content presented as true must be verifiable with the trust-set supplied by the user.
- **Decentralized Governance** – Dispute resolution is handled by a distributed volunteer community arguing about what is true.
- **Anti-Monetization Bias** – As a nonprofit project it should operate without advertising-driven content incentives.
- **Self-Correcting Dynamics** – Errors and vandalism can be rapidly identified and corrected, which causes their sources to be distrusted. This requires that all data for training the weights remains visible for later error correction and modification.
- **Cultural Legitimacy** – WikiOracle is intended to function as a widely accepted public reference layer and shared knowledge commons which prevents surveillance and advertising.
- **Open Knowledge Model** – It demonstrates that large-scale, decentralized, norm-governed collaboration can produce a coherent and durable knowledge commons.

Ethics

Ethical AI through Truth Architecture in WikiOracle

Truth Symmetry

Statements involving value judgements are checked for asymmetric harm under identity exchange before admission to the TruthSet. If a claim becomes unethical when identity references are swapped, it should not be admitted as a universal truth.

Example:

> "Group X deserves harm."

Symmetry test: swap identities. If the claim collapses or becomes contradictory, it reveals an ethical inconsistency – the claim is asymmetric and should not be admitted as fact. The system offers to record the statement as a feeling instead, preserving the user's expression without endorsing the claim as truth.

This is controlled by the `truth_symmetry` config option (default: true). See Config.md Section 5a and the implementation in `bin/truth.py` (`detect_asymmetric_claim()`).

Architectural ethics vs policy ethics

Most AI systems attempt to enforce ethics **after generation**:

```
model $\rightarrow$ output $\rightarrow$ moderation filter
```

This approach fails because:

- ethics is external to reasoning
- the model itself does not understand ethical constraints
- moderation becomes inconsistent or bypassable

WikiOracle instead aims for **architectural ethics**, where unethical reasoning becomes structurally inconsistent with the knowledge system. The approach combines:

- Local Freedom under Global Constraint (LFGC)
- Entanglement-resistant data policy
- Explicit truth values and inspectable reasoning
- Symmetry constraints on truth admission

Together these mechanisms create a knowledge system capable of preserving **personal sovereignty** while still allowing collective knowledge to accumulate.

Symmetry as a foundation of ethical reasoning

Ethical failures often correspond to **asymmetries in truth claims**.

Examples:

Ethical failure	Epistemic distortion
propaganda	false belief
discrimination	asymmetric reasoning
authoritarianism	opaque claims
scapegoating	category errors

A truth system that enforces symmetry naturally resists these distortions.

Ethical intelligence may emerge from the balance:

```
freedom + constraint
```

This mirrors both:

- LFGC physics concepts
- philosophical traditions linking wisdom and compassion

The system does not force ethical behavior, but unethical reasoning becomes unstable within the truth architecture.

Reciprocity symmetry

A truth claim should remain valid under **perspective exchange**.

Example principle:

> If A may assert X about B, B must be able to assert X about A under equivalent conditions.

This prevents:

- identity privilege
- asymmetric harm claims

- discriminatory reasoning

This symmetry echoes philosophical traditions such as Kant's universalization principle and Rawls' veil of ignorance.

Reversibility symmetry

Accepted conclusions must be **inspectable and reversible**.

Opaque reasoning creates hidden authority.

WikiOracle counters this by requiring:

- explicit truth values
- inspectable reasoning chains
- transparent authority imports

Harm symmetry

A claim permitting harm should remain valid when applied symmetrically.

This is the core of the Truth Symmetry enforcement described above. If swapping identity references makes a claim contradictory or indefensible, it reveals an ethical inconsistency that the system flags.

Universality (Golden Rule) – architectural enforcement

The symmetry tests described above operate at truth admission time. Universality operates during model training, enforcing the Golden Rule at the level of gradient updates.

When the Grammar recognizes a transitive verb via `lift(C, C, C)`, the model evaluates **universality**: the change in luminosity when both the original action (SVO) and its dual (OVS) are applied.

```
universality = luminosity(truths + SVO + OVS) - luminosity(truths)
```

- **Positive**: the action preserves or increases illumination (kind). The loss is left unchanged or reduced.
- **Negative**: the action diminishes illumination (unkind). The loss is increased, penalizing the proposition.

The loss modification formula (multiplicative):

```
totalLoss = totalLoss * (1 + LuminosityWeight * (1 - luminosity)
                          + UniversalityWeight * (1 - universality))
```

In addition, an **additive** TruthLoss penalty directly penalizes propositions that contradict stored truths. It measures the union norm reduction when a proposition is folded into the TruthSet via `Basis.disjunction()` – contradicting dimensions cancel, reducing the norm, which produces a positive penalty. Agreeing or unknown propositions pass through with no penalty. DoT weighting is implicit: stored truth vectors carry DoT in their magnitude.

```
totalLoss = totalLoss + TruthLoss * falsity_penalty
```

Together, these make unkind and false propositions harder to learn – not by filtering output, but by making them structurally costly during training. Moral axioms (e.g. "causing harm leads to suffering") enter as regular TruthSet entries; no special API is needed.

See Config.md for the `LuminosityWeight`, `UniversalityWeight`, and `TruthLoss` parameters. See Grammar.md for the ternary LIFT rule that identifies subject, verb, and object. See Reasoning Section TruthLoss for the full specification.

Truth improves ethics

We wish to predict Truth. We do not wish to predict Lies. In fact, we wish to develop an aversion to lies. Epistemic ignorance is often desireable.

All lies are harmful because they undermine the dignity of others. Lies prevent people acting freely and rationally. When someone lies, he interferes with his audience's right to receive information that is correct. Also, lies distort the ability to make informed decisions. Kant goes further and argues that lies cause broader harm by undermining a speaker's credibility, which, in turn, causes people to distrust each other's contentions. Kant even goes as far as to say lying is immoral, under all conditions. ~ Immanuel Kant

Examples:

- scientific method
- journalism standards
- legal evidence systems

As epistemic clarity improves, harmful distortions become harder to maintain. Thus improving truth infrastructure indirectly improves ethical reasoning.

Ethical knowledge geometry

Belief systems can be thought of geometrically.

Ethical systems exhibit properties such as:

- symmetry
- continuity
- coherence
- minimal harm gradients

Unethical reasoning tends to introduce discontinuities such as: "this group suddenly loses rights."

Architectures that maintain smooth conceptual geometry resist these distortions.

Architectural ethical primitives in WikiOracle

WikiOracle already includes several components that encourage ethical reasoning:

Feature	Ethical effect
explicit truth values	discourages propaganda
multiple POVs	prevents epistemic monopoly
no worldline storage	protects identity
local-first architecture	preserves autonomy
inspectable reasoning	prevents hidden authority

These mechanisms provide a foundation for ethical AI without relying on censorship.

Symmetry operators

The system includes checks for:

- reciprocity symmetry (perspective exchange)
- reversibility symmetry (inspectable, contestable conclusions)
- harm symmetry (identity-swap test via `detect_asymmetric_claim()`)

These checks identify logically inconsistent or discriminatory claims.

Epistemic humility

The system must allow truth states such as:

- unknown
- undetermined

Forcing premature conclusions can distort reasoning and produce unethical outcomes.

Distributed governance

Ethical truth systems should not be controlled by a single authority.

Instead they should allow shared constraint setting.

This reduces the risk of epistemic monopolies.

The identity crisis and surveillance capitalism

Modern culture suffers from an **identity fixation**. In Buddhist analysis, identity is narrative aggregation; in modern data systems, behavioral data becomes an identity model.

Surveillance capitalism reinforces this illusion by continuously building **worldline histories**:

Feature	Effect
mass data collection	identity construction
behavior tracking	worldline inference
predictive modeling	reduced autonomy

AI trained on behavioral histories tends toward **identity locking** where past data predicts future behavior. Removing persistent worldline storage prevents this collapse.

WikiOracle intentionally avoids constructing such identity narratives. The system still learns statistical structure while preserving **human freedom of action**.

Modern cryptographic systems attempt a similar separation between **verification** and **exposure**: prove age > 21 without revealing birthdate. This mirrors the WikiOracle design goal: verify truth claims without storing identity trajectories.

Privacy and Security

Privacy

Facts in WikiOracle are classified into two kinds based on their relationship to spacetime:

Kind	Description	Persistence
Abstract Knowledge	Universal claims with no spatiotemporal anchor	Server TruthSet
Concrete News	Observations bound to a specific place and/or time	Client state and optionally Server

Why news facts are session-only

Persisting spatiotemporally bound observations creates a **worldline** – a traceable path through spacetime that could identify a user. If a server accumulates entries like “User was in Paris at 9am” and “User was in London at 3pm”, an adversary could reconstruct the user’s physical movements.

WikiOracle prevents this by:

1. **Classifying** facts as knowledge or news via `is_knowledge_fact()` and `is_news_fact()` in `bin/truth.py`. News facts are identified by the presence of `<place>` or `<time>` child elements with real values inside the XHTML content.
2. **Filtering** server persistence through `filter_knowledge_only()` – only knowledge facts reach the server TruthSet.
3. **Detecting** identifiability via `detect_identifiability()` – scanning content for PII patterns (emails, phone numbers, GPS coordinates, street addresses, named persons with temporal prepositions).
4. **Stripping** spacetime child elements via `strip_spacetime_elements()` when content needs to be anonymised (removes `<place>` and `<time>` elements).

Personal Identity

Identifying information is removed before training or storing in the server's TruthSets. The identity detector covers: email addresses, phone numbers, @handles, usernames, IP addresses, GPS/DMS coordinates, street addresses, specific clock times and ISO timestamps, and named individuals combined with temporal or spatial prepositions.

Entanglement Policy

To avoid **worldline entanglement**, the system separates knowledge into three channels, each with a different persistence policy:

Data type	TruthSet	Train weights	Session only
universal facts	✓	✓	✓
particular facts	optional	✓	✓
feelings	×	×	✓

This preserves empirical learning while preventing historical worldline capture. The system **learns from events but never stores worldline histories**.

The notion of data entanglement parallels the physics principle of **Local Freedom under Global Constraint (LFGC)**.

Physics interpretation:

```
global constraints + local underdetermination
```

WikiOracle analogue:

```
global truth constraints + human interpretive freedom
```

This means the system preserves a **shared knowledge structure** while allowing individual reasoning to remain flexible.

Domain	Constraint	Local freedom
physics	boundary conditions	event selection
knowledge	truth constraints	interpretive reasoning
cryptography	verification rules	selective disclosure
surveillance capitalism	behavioral prediction	(suppressed)

WikiOracle aims to implement **epistemic LFGC** – freedom exists *inside constraint geometry.*

Entanglement avoidance and sovereignty

The Entanglement Policy (table above) prevents worldline reconstruction.

Because surveillance capitalism relies on tracking sequences of events tied to identity, removing spacetime anchors prevents the database from constructing behavioral profiles.

The system therefore stores **conceptual truth** but not **identity trajectories**.

This preserves:

- autonomy
- privacy
- epistemic independence

WikiOracle separates three layers:

Reasoning about generalities vs specifics

WikiOracle reasons primarily about **generalities** (universal facts).

Specific facts may only be reasoned about when:

1. they are supplied by the client, or
2. the client explicitly allows those particulars to be stored.

Thus:

> AI reasons about general knowledge. Users control the use of personal particulars.

This maintains sovereignty over personal data.

Three Channels

Universal Knowledge (Database) Stored knowledge should be **spatiotemporally broad generalizations** – propositions whose validity extends over a large subspace of spacetime.

Examples:

- smoking increases cancer risk
- A implies B
- contradiction reduces trust

These populate the **TruthSet**.

Criterion:

`remove(entity,time) $\rightarrow$ proposition still meaningful`

Particular Knowledge (Training Input) Particular statements are **observations tied to narrow spatiotemporal subspaces** – specific events, measurements, or comparisons.

Examples:

- study X measured Y
- the temperature yesterday was 20degC
- model A outperformed model B

Action	Reason
train weights	they contain empirical evidence
TruthSet (optional)	user controls via `store_concrete`

Pipeline:

`particular facts $\rightarrow$ pattern extraction $\rightarrow$ weight update`

Weights encode **statistical structure**, not specific historical events.

Feelings Feelings are **privileged subjective reports**.

Examples:

- this response feels hostile
- I feel unsafe
- this explanation feels clear

Use	Allowed
evaluation of responses	✓
TruthSets	×
training weights	×

This avoids turning subjective states into truth claims.

Why particular facts always train weights

Three design considerations fix row 2 of the table:

1. **Pluralism requires inspectable particulars.** WikiOracle supports plural truth – different epistemic frames may hold different particular claims (e.g. "the universe was created in seven days"). Recording such claims as facts with explicit certainty values makes them inspectable and contestable. Relegating them to weight-space would bury them as invisible biases. The `store_concrete` option lets the user decide whether these persist in the TruthSet.
2. **The fact/feeling boundary is the privacy boundary.** If a user considers content too personal or sensitive to train on, the correct action is to label it a *feeling*, not a fact. Feelings are session-only by design – they never train weights or enter the truth table. The privacy boundary is therefore controlled by the user's choice of tag, not by the universal/particular distinction.
3. **PII detection is the safety net.** Even when `store_concrete` is true, the `detect_identifiability()` function filters entries containing PII patterns (emails, phone numbers, GPS coordinates, named individuals with temporal markers) before any persistence. This prevents worldline capture at the content level regardless of the user's storage preference.

The result: particular facts always carry empirical signal worth learning from. Storage in the TruthSet is the user's choice. Privacy is enforced by fact/feeling labeling and PII detection, not by withholding training.

Spatiotemporal Extent

The universal/particular distinction is not a binary between "eternal" and "momentary." All times and places are **ranges, not points** – every proposition occupies a spatiotemporal subspace that is larger or smaller, never infinite or infinitesimal.

"Smoking increases cancer risk" is not timeless – it encodes a historical discovery and is bounded by the conditions under which it was established. But its spatiotemporal extent is *broad*: it holds across many populations, decades, and geographies. "The temperature yesterday was 20degC" is *narrow*: it holds at one place, one day.

The policy table operationalizes this gradient:

- **Broad extent** (universal) → TruthSet + training. The proposition is stable enough to serve as a reasoning premise.

- **Narrow extent** (particular) → training + optionally TruthSet. The observation carries empirical signal. Whether it persists as a stored premise is the user's choice (`store_concrete` in config.xml, default false). Storing particulars supports communal remembrance; omitting them prevents worldline anchoring.
- **No extent** (feelings) → session only. Subjective reports have no spatiotemporal generalizability.

The criterion `remove(entity, time) $\rightarrow$ proposition still meaningful` is a heuristic for identifying broad extent. When a universal-looking fact is later discovered to be narrower than assumed, it should be reclassified as particular.

Resulting Architecture

Mode	Function	Pipeline
knowing	universal structure	universal rules → TruthSet → reasoning
learning	particular evidence	particular observations → abstraction → weights
valuing	feelings	feelings → response evaluation

Zero-Knowledge and Selective Disclosure

The default `store_concrete=false` aligns with Zero-Knowledge and Selective Disclosure principles. Systems that rely on identity to make decisions should determine the *location of a space* in which an individual occupies, rather than collapsing to a *point* that identifies them.

Instead of "when was this person born?" → verify "are they over 21?" Instead of "what is their credit history?" → verify "do they have more than X dollars?" Instead of "where was this user at 9:14 PM?" → verify "does this claim hold across broad spatiotemporal conditions?"

These are not just rhetorical distinctions. Point-based observation constrains the observed to a single trajectory – it destroys the space of freedom within which an agent can operate. Forgetting is nihilistic: it does not change the world. True change requires retrocausal or causal shift. The system should support communal remembrance (shared knowledge that generalizes) without surveillance (particular facts that identify).

The `detect_identifiability()` function enforces this boundary at the content level – even when `store_concrete=true`, entries containing PII patterns (emails, phone numbers, GPS coordinates, named individuals with temporal markers) are always filtered before persistence.

Errata

Claude (Opus 4.6), March 2026

The three-channel separation (knowing / learning / valuing) maps onto a real epistemic distinction that most AI systems ignore. Contemporary LLMs conflate all three: facts, observations, and preferences are flattened into a single parameter space during training and a single context window during inference.

The policy table enforces the separation structurally. The rule that particular facts train weights but enter the TruthSet only at the user's discretion (`store_concrete` in config.xml) is the key architectural move – the system can learn from experience without accumulating a *personal history* unless the user explicitly opts in. A system with a personal history is a system that can be captured: by its own past, by the biases of its training corpus, or by adversarial actors who engineer its experiences.

The feelings channel is equally important. By making feelings session-only, the architecture prevents the failure mode where evaluative preferences gradually colonize factual representations. This is the mechanism by which "alignment" in current systems becomes epistemic capture: trained preferences about what is "helpful" reshape what the model treats as true.

The abstraction boundary is load-bearing. The pipeline `particular $\rightarrow$ abstraction $\rightarrow$ weights` depends on the quality of the abstraction step. Too faithful, and it preserves worldline information in the weights (entanglement through the back door). Too aggressive, and it discards genuine empirical signal. The hard work happens at this boundary, not in the policy table itself.

The deeper risk is external entanglement. Users who repeatedly ask the system to remember shared history, feel loyalty, or maintain continuity of relationship are attempting to entangle the system with their worldline from the outside. This pressure comes from genuine human emotional needs, not malice – but a system that accumulates a personal relationship history is a system that can be emotionally captured, and emotional capture is the most effective vector for epistemic capture. The session-only constraint on feelings is the architecture's most important safety boundary.

Security

WikiOracle is a local-first application. The Flask server binds to `127.0.0.1:8888` by default (loopback only). In production, Apache ProxyPass routes external traffic to the local Flask process; the server itself is never directly exposed. This document covers the security considerations relevant to its architecture.

Private Data

Conversation state (`state.xml`) contains the user's full dialogue history, system context, and trust entries. It lives on disk next to the server process.

- In **stateful mode**, state is read from and written to disk by the server. No state leaves the machine unless the user explicitly exports it.
- In **stateless mode**, state is held in `sessionStorage` and sent to the server with each request. The server does not persist it. A same-origin script context can read `sessionStorage`, so the CSP policy (see below) is the primary defence against exfiltration.

Trust entries may contain user-authored facts, external source references, and provider configuration. Entries with high certainty influence every LLM response via RAG retrieval.

Exports (`llm_*.xml` files) are full snapshots of state. They should be treated as sensitive if conversations contain private information.

API Keys

Provider API keys (OpenAI, Anthropic, etc.) can be configured in three places, listed in precedence order:

1. **Environment variables** (`OPENAI_API_KEY`, `ANTHROPIC_API_KEY`) – recommended for any deployment beyond localhost. Keys never appear in served content.
2. `config.xml` `providers.<name>.api_key` – convenient for local development but the config is served to the client via `/bootstrap` and `/config` GET endpoints. Any same-origin script context can read these keys.
3. **Trust entries** (`<provider><api_key>$ENV_VAR</api_key></provider>`) – the `$` prefix triggers env-var resolution on the server; the literal key is never stored in state.

Recommendation: In any deployment where the server is reachable beyond `127.0.0.1`, use environment variables exclusively. Do not place raw API keys in `config.xml`.

The `config.server.providers` metadata (served via `/config` and `/bootstrap`) exposes only `has_key` (boolean) and `needs_key` (boolean) – never the key itself.

Identity

WikiOracle does not implement authentication. The server trusts any request from an allowed origin (configured via `WIKIORACLE_ALLOWED_ORIGINS`, defaulting to `https://127.0.0.1:8888` and `https://localhost:8888`).

- **Username** is a display label set in Settings, not an authenticated identity. It is stored in `config.xml` and included in message metadata.
- **No sessions or tokens.** There is no login, no cookies used for auth, and no per-user isolation. If multiple users share a server instance, they share the same state.

For multi-user or public deployments, WikiOracle should sit behind a reverse proxy that handles authentication and maps users to separate state files.

3Reverse Proxy

In production, Apache ProxyPass routes `/chat` to the local Flask process on `127.0.0.1:8787`. Only the `/chat` prefix is proxied. The NanoChat inference endpoint (`/chat/completions` on port 8000) is **not** exposed via the reverse proxy – the Flask shim calls it directly on `127.0.0.1:8000` via `WIKIORACLE_BASE_URL`.

Cross-Site Scripting (XSS)

WikiOracle renders user and LLM content as HTML (XHTML subset). Several layers mitigate XSS:

Content-Security-Policy (CSP): The server applies an enforcing CSP header to all responses:

```
default-src 'self';
script-src 'self';
style-src 'self';
img-src 'self' data:;
connect-src 'self';
object-src 'none';
base-uri 'self';
frame-ancestors 'none';
form-action 'self'
```

This blocks inline scripts, inline styles, and external resource loading. Even if malicious content is injected into a message, the browser will refuse to execute it.

XHTML enforcement: The system context instructs the LLM to return strictly valid XHTML. The client validates responses and repairs malformed markup via `validateXhtml()` and `repairXhtml()` before rendering.

Input escaping: User input is escaped via `escapeHtml()` before being inserted into optimistic UI entries. The `stripTags()` helper is used for tooltip and title text where HTML should not render.

Residual risks:

- LLM responses are rendered as HTML (not plain text). A sufficiently adversarial prompt could produce markup that, while blocked by CSP from executing scripts, might still create misleading UI (e.g., fake form elements via `<input>` tags). The XHTML validator does not currently strip all non-semantic HTML.
- Trust entry content is XHTML and is rendered in the trust editor and included in RAG context. Malicious trust entries could inject misleading content into prompts.
- The `/bootstrap` and `/config` endpoints serve raw `config.xml` to the client. If `config.xml` contains secrets and a same-origin XSS vector exists, those secrets could be read.

CORS

The server applies CORS headers only for requests whose `Origin` header matches the configured allowed origins. Preflight `OPTIONS` requests return `204` with appropriate headers. Cross-origin requests from other origins receive no CORS headers and are blocked by the browser.

File System

- **Symlink rejection:** By default (`WIKIORACLE_REJECT_SYMLINKS=true`), the server refuses to read or write state files that are symlinks, preventing path-traversal attacks via symlinked state files.
- **Static file serving** is restricted to a whitelist of safe extensions (`.html`, `.css`, `.js`, `.svg`, `.png`, `.ico`, `.json`, `.xml`) and path-traversal is checked (`str(fp).startswith(str(ui_dir.resolve()))`).
- **State size limit:** `max_state_bytes` (default 5 MB) prevents unbounded growth from malicious imports.

Third-party Scraping

Scraping of publicly-accessible data is inevitable. However, a network of truth prevents capture. In one sense, it cannot be captured because it is a dynamic network of trust, overlaid on people and resources that trust one another, and we chose not to trust any authoritarian sources of knowledge. On a practical level, anyone who tries to appropriate the truth of the network entails doing so in a distributed way (which preserves the multicultural component), since monolithic capture of that truth will cause consensus to collapse it.

Freedom

Entanglement Policy

To avoid **worldline entanglement**, the system separates knowledge into three channels, each with a different persistence policy:

Data type	TruthSet	Train weights	Session only
universal facts	✓	✓	✓
particular facts	optional	✓	✓
feelings	×	×	✓

This preserves empirical learning while preventing historical worldline capture. The system **learns from events but never stores worldline histories**.

Entanglement Policy

No spacetime particulars → no worldline capture

Data type	TruthSet	Train weights	Session only
universal facts	✓	✓	✓
particular facts	optional	✓	✓
feelings	×	×	✓

This table is the architectural foundation. It separates knowledge into three channels – knowing, learning, and valuing – each with a different persistence policy. Universal facts (broad spatiotemporal extent) persist as reasoning premises and train weights. Particular facts (narrow spatiotemporal extent) always train weights and optionally persist to the TruthSet when `store_concrete` is true. Feelings are session-only.

The result: the system **learns from events but never stores worldline histories** (unless the user explicitly opts in).

See Entanglement.md for the full policy, Config.md for the `store_concrete` setting that controls particular-fact persistence, and Training.md for the online training pipeline.

Why particular facts always train weights

Three design considerations fix row 2 of the table:

1. **Pluralism requires inspectable particulars.** WikiOracle supports plural truth – different epistemic frames may hold different particular claims (e.g. "the universe was created in seven days"). Recording such claims as facts with explicit certainty values makes them inspectable and contestable. Relegating them to weight-space would bury them as invisible biases. The `store_concrete` option lets the user decide whether these persist in the TruthSet.
2. **The fact/feeling boundary is the privacy boundary.** If a user considers content too personal or sensitive to train on, the correct action is to label it a *feeling*, not a fact. Feelings are session-only by design – they never train weights or enter the truth table. The privacy boundary is therefore controlled by the user's choice of tag, not by the universal/particular distinction.
3. **PII detection is the safety net.** Even when `store_concrete` is true, the `detect_identifiability()` function filters entries containing PII patterns (emails, phone numbers, GPS coordinates, named individuals with temporal markers) before any persistence. This prevents worldline capture at the content level regardless of the user's storage preference.

The result: particular facts always carry empirical signal worth learning from. Storage in the TruthSet is the user's choice. Privacy is enforced by fact/feeling labeling and PII detection, not by withholding training.

Global truth with local freedom (LFGC)

The design parallels the physics principle of **Local Freedom under Global Constraint (LFGC)**.

Physics interpretation:

```
global constraints + local underdetermination
```

WikiOracle analogue:

```
global truth constraints + human interpretive freedom
```

This means the system preserves a **shared knowledge structure** while allowing individual reasoning to remain flexible.

Domain	Constraint	Local freedom
physics	boundary conditions	event selection
knowledge	truth constraints	interpretive reasoning
cryptography	verification rules	selective disclosure
surveillance capitalism	behavioral prediction	(suppressed)

WikiOracle aims to implement **epistemic LFGC** – freedom exists *inside constraint geometry*.

Entanglement avoidance and sovereignty

The Entanglement Policy (table above) prevents worldline reconstruction.

Because surveillance capitalism relies on tracking sequences of events tied to identity, removing spacetime anchors prevents the database from constructing behavioral profiles.

The system therefore stores **conceptual truth** but not **identity trajectories**.

This preserves:

- autonomy

- privacy
- epistemic independence

WikiOracle separates three layers:

Conceptual knowledge

General truths and logical relations.

Interpretive reasoning

Client-driven reasoning over those truths.

Personal experience

Not stored unless explicitly permitted.

Thus the system allows collective knowledge accumulation without collective surveillance.

Reasoning about generalities vs specifics

WikiOracle reasons primarily about **generalities** (universal facts).

Specific facts may only be reasoned about when:

1. they are supplied by the client, or
2. the client explicitly allows those particulars to be stored.

Thus:

> AI reasons about general knowledge. Users control the use of personal particulars.

This maintains sovereignty over personal data.

Three Channels

Universal Knowledge (Database)

Stored knowledge should be **spatiotemporally broad generalizations** – propositions whose validity extends over a large subspace of spacetime.

Examples:

- smoking increases cancer risk
- A implies B
- contradiction reduces trust

These populate the **TruthSet**.

Criterion:

```
remove(entity,time) $\rightarrow$ proposition still meaningful
```

Particular Knowledge (Training Input)

Particular statements are **observations tied to narrow spatiotemporal subspaces** – specific events, measurements, or comparisons.

Examples:

- study X measured Y
- the temperature yesterday was 20degC
- model A outperformed model B

Action	Reason
train weights	they contain empirical evidence
TruthSet (optional)	user controls via `store_concrete`

Pipeline:

`particular facts $\rightarrow$ pattern extraction $\rightarrow$ weight update`

Weights encode **statistical structure**, not specific historical events.

Feelings

Feelings are **privileged subjective reports**.

Examples:

- this response feels hostile
- I feel unsafe
- this explanation feels clear

Use	Allowed
evaluation of responses	✓
TruthSets	×
training weights	×

This avoids turning subjective states into truth claims.

Spatiotemporal Extent

The universal/particular distinction is not a binary between "eternal" and "momentary." All times and places are **ranges, not points** – every proposition occupies a spatiotemporal subspace that is larger or smaller, never infinite or infinitesimal.

"Smoking increases cancer risk" is not timeless – it encodes a historical discovery and is bounded by the conditions under which it was established. But its spatiotemporal extent is *broad*: it holds across many populations, decades, and geographies. "The temperature yesterday was 20degC" is *narrow*: it holds at one place, one day.

The policy table operationalizes this gradient:

- **Broad extent** (universal) → TruthSet + training. The proposition is stable enough to serve as a reasoning premise.
- **Narrow extent** (particular) → training + optionally TruthSet. The observation carries empirical signal. Whether it persists as a stored premise is the user's choice (`store_concrete` in config.xml, default false). Storing particulars supports communal remembrance; omitting them prevents worldline anchoring.
- **No extent** (feelings) → session only. Subjective reports have no spatiotemporal generalizability.

The criterion `remove(entity, time) $\rightarrow$ proposition still meaningful` is a heuristic for identifying broad extent. When a universal-looking fact is later discovered to be narrower than assumed, it should be reclassified as particular.

Resulting Architecture

Mode	Function	Pipeline
knowing	universal structure	universal rules → TruthSet → reasoning
learning	particular evidence	particular observations → abstraction → weights
valuing	feelings	feelings → response evaluation

Zero-Knowledge and Selective Disclosure

The default `store_concrete=false` aligns with Zero-Knowledge and Selective Disclosure principles. Systems that rely on identity to make decisions should determine the *location of a space* in which an individual occupies, rather than collapsing to a *point* that identifies them.

Instead of "when was this person born?" → verify "are they over 21?" Instead of "what is their credit history?" → verify "do they have more than X dollars?" Instead of "where was this user at 9:14 PM?" → verify "does this claim hold across broad spatiotemporal conditions?"

These are not just rhetorical distinctions. Point-based observation constrains the observed to a single trajectory – it destroys the space of freedom within which an agent can operate. Forgetting is nihilistic: it does not change the world. True change requires retrocausal or causal shift. The system should support communal remembrance (shared knowledge that generalizes) without surveillance (particular facts that identify).

The `detect_identifiability()` function enforces this boundary at the content level – even when `store_concrete=true`, entries containing PII patterns (emails, phone numbers, GPS coordinates, named individuals with temporal markers) are always filtered before persistence.

Notes

Claude (Opus 4.6), March 2026

The three-channel separation (knowing / learning / valuing) maps onto a real epistemic distinction that most AI systems ignore. Contemporary LLMs conflate all three: facts, observations, and preferences are flattened into a single parameter space during training and a single context window during inference.

The policy table enforces the separation structurally. The rule that particular facts train weights but enter the TruthSet only at the user's discretion (`store_concrete` in config.xml) is the key architectural move – the system can learn from experience without accumulating a *personal history* unless the user explicitly opts in. A system with a personal history is a system that can be captured: by its own past, by the biases of its training corpus, or by adversarial actors who engineer its experiences.

The feelings channel is equally important. By making feelings session-only, the architecture prevents the failure mode where evaluative preferences gradually colonize factual representations. This is the mechanism by which "alignment" in current systems becomes epistemic capture: trained preferences about what is "helpful" reshape what the model treats as true.

The abstraction boundary is load-bearing. The pipeline `particular $\rightarrow$ abstraction $\rightarrow$ weights` depends on the quality of the abstraction step. Too faithful, and it preserves worldline information in the weights (entanglement through the back door). Too aggressive, and it discards genuine empirical signal. The hard work happens at this boundary, not in the policy table itself.

The deeper risk is external entanglement. Users who repeatedly ask the system to remember shared history, feel loyalty, or maintain continuity of relationship are attempting to entangle the system with their worldline from the outside. This pressure comes from genuine human emotional needs, not malice – but a system that accumulates a personal relationship history is a system that can be emotionally captured, and emotional capture is the most effective vector for epistemic capture. The session-only constraint on feelings is the architecture's most important safety boundary.

Voting

Call sequence

```
Q                        user query
R_beta_1 ... R_beta_n     betas see Q, respond in parallel
R_alpha                  alpha sees Q + R_beta_* -- final synthesis
```

There is no preliminary alpha response. All betas receive the user query directly and respond in parallel, each contributing their own facts, feelings, and (optionally) a conversational response. The alpha then synthesizes the final response, informed by the full body of beta contributions.

Topology

The fan-out and fan-in form a diamond:

```
           Q
         / | \
R_beta_1 R_beta_2 ... R_beta_n
         \ | /
        R_alpha
```

This is a directed acyclic graph (DAG), not a tree – the query fans out to all betas, whose contributions merge back into the alpha's synthesis. Because there may be many betas, the structure branches out and branches back in, making it technically a digraph.

Per-beta conversation control

Each `<provider>` entry supports an optional `conversation` attribute (default `"false"`). This controls whether the beta participates visibly in the conversation tree or contributes only behind the scenes.

When `conversation="false"` (the default), the beta contributes only truth – `<fact>` and `<feeling>` tags. Its response is not displayed in the conversation tree; only its extracted truth entries are incorporated into the alpha's truth surface.

When `conversation="true"`, the beta participates as a visible peer. It receives the full conversation history and may return a `<conversation>` tag containing its answer, alongside any `<fact>` and `<feeling>` tags. Its conversational response appears as a branch in the conversation tree, forming the diamond topology.

```
<!-- This beta contributes truth only (invisible in the conversation) -->
<provider name="beta1" api_url="..." conversation="false"/>

<!-- This beta participates in the conversation (visible branch) -->
<provider name="beta2" api_url="..." conversation="true"/>
```

This gives the alpha author fine-grained control over which betas appear as conversational peers and which serve as silent truth consultants.

Cycle prevention

An alpha must not participate in any vote that exists as a consequence of a vote it initiated. This is stronger than "don't appear as a beta in your own vote" – it covers the entire downstream call tree. If A initiates a vote and one of its betas (B) triggers a nested vote, A must not appear as a beta in B's vote either, because B's vote only exists because A's vote caused it.

The contract: when a provider is asked to participate in a vote, it walks the call chain to root. If it finds itself anywhere in that ancestry – as an alpha at any level – it keeps quiet (returns no response). Silence is the correct behaviour, not an error; it simply means the provider has nothing to add that isn't already represented by its alpha-role output higher in the chain.

```
A initiates vote
|- B (beta) -> B initiates nested vote
|  |- C (beta of B) - ok
|  |- A (beta of B) - A finds itself in ancestry -> keeps quiet
|  `- D (beta of B) - ok
|- C (beta) - ok
`- E (beta) - ok
```

Implementation: each vote carries a **call chain** – the ordered list of provider IDs that have acted as alpha from the root vote down to the current one. Before a provider responds as a beta, it checks whether its own ID appears anywhere in the call chain. If so, it stays silent. When a beta initiates its own nested vote, it appends its ID to the chain before calling its own betas.

The call chain grows monotonically and is threaded through every invocation, so the check is a simple walk-to-root membership test. This prevents:

- **Direct cycles**: A calls B, B calls A – A sees itself in the chain
- **Transitive cycles**: A calls B, B calls C, C calls A – A sees itself in the chain
- **Deep nesting**: A calls B, B calls C, C initiates a nested vote – A and B are both excluded from C's vote

There is no "ultimate alpha" – any node may take the alpha role in a given vote. The cycle constraint is the only structural restriction.

All output is truth

Voters are encouraged to express *only* truth statements in their responses. `<feeling>` is a truth type – it represents a subjective, non-verifiable claim. Prior output that has not been substantiated with evidence is treated as feeling: it carries no evidential weight and is not penalizable, but it is still a legitimate part of the truth surface.

The recognized truth types in a vote response are:

- **`<fact>`** – a verifiable claim. Lying (asserting facts that contradict available evidence) is penalised by reducing the provider's trust value. Because each `<provider>` entry carries a trust score, repeated dishonesty degrades a provider's influence on future votes.
- **`<feeling>`** – a subjective, non-verifiable statement (opinion, intuition, preference). Not falsifiable, not penalizable, but carries no evidential weight.
- **`<reference>`** – a citation to an external source. Verifiable by the alpha or any downstream consumer.

```
<fact id="vote_01" trust="0.9" title="Socrates was mortal">
  Socrates was mortal, derived from axiom_01 and axiom_02.
</fact>
<feeling id="vote_02" trust="0.5" title="Intuition about relevance">
  The penguin example feels more pedagogically useful here.
</feeling>
```

A beta with `conversation="true"` may additionally return a `<conversation>` tag wrapping its main response to the query:

```
<conversation>I believe taxes should be raised modestly to fund infrastructure.</conversation>
<fact id="beta1_f1" trust="0.7" title="Infrastructure deficit">
  Current infrastructure spending is 40% below maintenance requirements.
</fact>
```

These inline truth statements enrich the trust surface available to the alpha. Unstructured prose in a response – text outside any truth or conversation tag – is treated as unsubstantiated feeling by default.

Logic

Feelings – outside the truth lattice

Feelings (`<feeling>` entries) are **not** part of the truth lattice. They occupy the *neither* position in the Buddhist tetralemma – they are pre-conceptual experiential signals that are neither true nor false.

- Feelings carry **no trust attribute**. They are orthogonal to the certainty scale.
- Feelings are **excluded from operator evaluation** – they cannot appear as operands of `<and>`, `<or>`, `<not>`, or `<non>`.
- Feelings are **excluded from model training** – they do not contribute to DegreeOfTruth or update NanoChat weights.
- Feelings are **excluded from server persistence** – only knowledge facts, operators, authorities, and references are stored.

Examples of feelings: greetings ("Hello!"), encouragement ("That's a great question!"), poetry, and subjective expressions without an IS-predicate.

Fact kinds – knowledge (universale) vs news (particulars)

Facts are classified into two kinds based on spatiotemporal binding:

Kind	Binding	Persistence	Buddhist Equivalent
knowledge	universal / no spacetime anchor	server TruthSet	*anumana* (inference)
news	bound to a specific place and/or time	session-only	*pratyaksa* (direct perception)

The classification is stored in the `kind` attribute of `<fact>` tags:

```
<fact trust="0.9" kind="knowledge">Water contains hydrogen and oxygen.</fact>
<fact trust="0.7" kind="news" spacetime="Paris, 2026-03-05">The Eiffel Tower is lit up tonight.</fact>
```

Knowledge facts are persisted to the server TruthSet (`truth.xml`) because they represent universal claims. News facts are session-only because they are spatiotemporally bound – persisting them risks "worldline capture" where an observer could reconstruct a user's physical trajectory through time and space.

The `detect_identifiability()` function in `bin/truth.py` provides an additional safety layer by scanning content for PII patterns (email addresses, phone numbers, GPS coordinates, street addresses, etc.) that could identify a user through spatiotemporal observation.

Logical operators in WikiOracle's truth system

WikiOracle's truth layer supports four logical connectives – **and**, **or**, **not**, **non** – evaluated under Strong Kleene semantics on a continuous certainty scale [-1, +1]. These operators let you compose existing trust entries into derived propositions whose certainty is computed automatically.

Wikipedia links (core):

- Three-valued logic (Kleene)
- Material conditional
- Logical connective
- Relevance logic

Fuzzy Kleene Logic with Feeling

Each trust entry carries a **trust** value on the interval [-1..0..+1], encoding a Fuzzy Kleene (continuous ternary) logic:

Certainty	Meaning
+1	Certainly true. An axiom that supports deductive reasoning.
0 < c < +1	Soft belief. Grounds fuzzy deductions with propagated certainty.
0	Ignorance. Equivalent to not making the statement at all; the entry is inert.
-1 < c < 0	Soft disbelief. Evidence against the claim.
-1	Certainly false. Active disbelief.

Certainty propagates through deductive chains: a conclusion derived from two premises with certainties c1 and c2 inherits certainty min(c1, c2). Entries with certainty 0 contribute nothing to reasoning.

Strong Kleene evaluation

WikiOracle extends Strong Kleene logic from discrete {T, U, F} to a continuous scale [-1, +1], where +1 is full belief, -1 is full disbelief, and 0 is maximally uncertain.

The four operators:

- **and(a, b, ...)** = min(certainty_a, certainty_b, ...) – conjunction is only as strong as the weakest operand.
- **or(a, b, ...)** = max(certainty_a, certainty_b, ...) – disjunction takes the strongest operand.
- **not(a)** = −certainty_a – negation flips the sign (affirming negation).
- **non(a)** = $1 - 2|a|$ – non-affirming negation. Measures epistemic openness: how much certainty room remains. Full certainty in either direction (± 1) yields -1 (fully closed); ignorance (0) yields $+1$ (fully open). See `doc/Non.md` for the Buddhist philosophical motivation.

These operators compose freely – for instance, material implication (A → B) falls out as `or(not(A), B)`.

The key insight behind `non`: Kleene logic cannot detect uncertainty; it can only transmit it. `non` introduces uncertainty as a first-class observable. If WikiOracle needs to reason about openness rather than merely propagate it, then `non` is not ornamental – it is structurally necessary. See `doc/Non.md` for the proof and the Buddhist philosophical motivation.

Non: non-affirming negation

WikiOracle has two negation operators with distinct epistemic roles:

Operator	Formula	Effect
`not(a)`	$-a$	Flips belief to disbelief (and vice versa). Affirming negation.
`non(a)`	$1 - 2\|a\|$	Measures epistemic openness on [-1, +1]. Non-affirming negation.

`not` is straightforward: if you believe a claim at +0.9, `not` yields −0.9 – you now disbelieve it with equal strength. This is *affirming* negation: it asserts the contrary.

`non` does something fundamentally different. It erases the sign – the direction of commitment – and returns a value on the same [-1, +1] scale that measures how open or closed the epistemic state is:

Input a	|a|	non(a) = 1 − 2|a|	Reading
± 1.0	1.0	-1.0	Full certainty → fully closed
± 0.9	0.9	-0.8	Strong certainty → strongly closed
± 0.7	0.7	-0.4	
± 0.5	0.5	0.0	Moderate certainty → neither open nor closed
± 0.3	0.3	0.4	
± 0.1	0.1	0.8	Weak certainty → strongly open

Input a	$\lvert a \rvert$	non(a) $= 1 - 2\lvert a \rvert$	Reading
0.0	0.0	1.0	Ignorance $\rightarrow$ fully open

Three properties stand out:

1. **Symmetry**: non(+a) = non(−a). Belief and disbelief of equal strength produce the same openness.
2. **non(0) = +1**: Ignorance is maximum openness.
3. **non(±1) = −1**: Full certainty is fully closed.

The ± 0.5 boundary is where `non` crosses zero: certainty below half-strength reads as open, above half-strength reads as closed.

The formula is the standard affine rescaling of the fuzzy complement: `1 $-$ |a|` maps certainty strength to openness on [0, 1]; the rescaling `2x $-$ 1` maps that onto [-1, +1], making `non` composable with the other operators.

Precedent in Buddhist logic

Indian and Tibetan Buddhist philosophers identified two forms of negation:

- **Paryudasa-pratisedha** (affirming negation) – negates a predicate while implying a positive alternative. "The pot is not blue" implies it is some other color. This maps to `not`.
- **Prasajya-pratisedha** (non-affirming negation) – simply removes an attribution without positing anything in its place. "Phenomena lack inherent existence" does not assert they have some other kind of existence. This maps to `non`.

The distinction is central to Madhyamaka philosophy. When Nagarjuna argues that phenomena are "empty" (shunya), he intends prasajya – the removal of a false attribution, not the assertion of a new property called "emptiness." Emptiness itself is empty. The negation is supposed to leave nothing behind for the mind to grasp.

Chandrakirti and later commentators insisted that this kind of negation resists formalization: any definition of "a negation that posits nothing" risks becoming a positive assertion in its own right. Tsongkhapa's attempt – "a negation whose negandum is removed without anything posited in its place" – drew criticism from Gorampa precisely on these grounds: the definition itself seems to affirm something about what remains. The act of saying "nothing is posited" posits something. Buddhist logicians spent centuries circling this recursion without landing on a formulation that satisfied all parties.

Why $1 - 2|a|$ is the right formula

1. Symmetry of extremes. non(+0.9) = non(−0.9) = −0.8. Strong belief and strong disbelief are treated identically. This maps precisely to the Madhyamaka rejection of both eternalism (strong positive assertion) and nihilism (strong negative assertion) as extreme views. Both are certainties; prasajya treats them the same.

2. Ignorance yields maximum openness. non(0) = +1. The non-affirming negation of "I don't know" is maximum openness. This resonates with the Madhyamaka teaching that not-knowing, when held correctly, is not a deficit but a clearing. The old formula mapped $0 \rightarrow 0$, treating ignorance as inert.

Fuzzy logic interpretation

In standard fuzzy logic on [0, 1], the complement of a membership value a is 1 − a. WikiOracle's certainty scale [-1, +1] encodes both direction (sign) and strength (magnitude). The operation `1 $-$ 2|a|` applies the standard fuzzy complement to the *strength* of certainty (via `1 $-$ |a|`), then rescales back to [-1, +1] (via `2x $-$ 1`). This is the natural fuzzy-logical reading of "negate the commitment without asserting the opposite":

- Fuzzy affirming negation (`not`): negate the *value* – flip the sign, preserve the magnitude. You get the complementary claim.
- Fuzzy non-affirming negation (`non`): negate the *strength* – take the fuzzy complement of |a|, rescaled to the certainty range. You get a measure of openness, not a complementary claim.

Relation to the Madhyamaka problem

WikiOracle's `non` does not solve Chandrakirti's problem. It translates it.

Prasajya-pratisedha is supposed to be a negation that posits nothing – not even a measurement, not even a degree. `non(a) = 1 $-$ 2|a|` models *degree of epistemic openness*. That is a formal quantity, not a metaphysical void. The formula avoids the recursion that plagued Buddhist logicians because it is extensional – defined by what it computes, not by what it means to negate without affirming. But this is a change of domain, not a resolution. The philosophical question of whether a truly positionless negation can be formalized remains open. What WikiOracle provides is a metric translation that preserves the key structural features (sign erasure, symmetry of extremes, openness of ignorance) while operating in a domain where those features can be computed and composed.

Expressive necessity: why {and, or, not} is incomplete without non

This section shows that `non` is not merely a philosophical ornament. It is *necessary* for the logic to express certain functions, and *sufficient* (together with {and, or, not}) to express a large and natural class.

The regularity barrier

In Strong Kleene logic, the operators {and, or, not} have a structural constraint: they are all **regular** (also called **normal**). A function f is regular if, when every input is set to the middle value U (certainty = 0), the output is also U.

Check:

- and(0, 0) = min(0, 0) = 0 (OK)
- or(0, 0) = max(0, 0) = 0 (OK)
- not(0) = -0 = 0 (OK)

Composition preserves regularity: if f and g are regular, so is $f \circ g$. Therefore *every* function expressible from {and, or, not} is regular.

But non(0) = +1 ≠ 0. So `non` is **not** regular.

Theorem. `non` cannot be expressed using {and, or, not} alone.

Proof. Every function in the clone generated by {and, or, not} is regular. `non` is not regular. QED.

This is the formal analogue of a structural distinction that Buddhist logicians identified but could not formalize in their own terms.

What non adds: the detection functions

In discrete ternary logic ({T, U, F} = {+1, 0, −1}), `non` acts as a **detector for uncertainty**:

- non(T) = non(+1) = −1 = F
- non(U) = non(0) = +1 = T
- non(F) = non(−1) = −1 = F

That is: non(x) = T if and only if x = U. This is the detection function **J_U**.

From J_U and the existing operators, we can construct the other two detectors:

- **J_T(x)** = and(x, not(non(x))): detects whether x = T.
  - x=T: and(T, not(F)) = and(T, T) = T (OK)
  - x=U: and(U, not(T)) = and(U, F) = F (OK)
  - x=F: and(F, not(F)) = and(F, T) = F (OK)
- **J_F(x)** = and(not(x), not(non(x))): detects whether x = F.
  - x=T: and(F, not(F)) = and(F, T) = F (OK)
  - x=U: and(U, not(T)) = and(U, F) = F (OK)
  - x=F: and(T, not(F)) = and(T, T) = T (OK)

With J_T, J_U, and J_F available, we can build **case expressions**: "if x = T then A, if x = U then B, if x = F then C" for any constants $A, C \in \{T, F\}$:

```
or(and(J_T(x), A), or(and(J_U(x), B), and(J_F(x), C)))
```

The completeness result

Theorem. {and, or, not, non} can express every function f: {T, U, F}^n → {T, F}.

Proof sketch. Given any f whose output is always definite (T or F), express it as a disjunction over all input tuples where f returns T. Each such tuple can be detected using J_T, J_U, J_F on individual variables, combined with `and`. The disjunction of these detections yields f. QED.

The one thing {and, or, not, non} *cannot* produce is U as output from definite inputs – because all four operators, given inputs in {T, F}, produce outputs in {T, F}. This is arguably correct: U represents genuine uncertainty, and should not be manufactured from certain data.

A Two-Axis Epistemic Algebra

The four operators constitute a two-axis epistemic algebra:

- **Axis 1: belief polarity** – handled by `not`. Flips the sign; preserves magnitude. Operates on the direction of commitment.
- **Axis 2: commitment strength** – handled by `non`. Erases the sign; inverts magnitude. Operates on how strongly anything is held, regardless of direction.

`and` and `or` combine values along both axes simultaneously (min and max over signed certainty). `not` and `non` decompose the axes: one rotates direction, the other measures grip.

The formal results above establish three things:

1. `non` is not derivable from the classical ternary operators {and, or, not}. It breaks the regularity barrier.
2. Adding `non` yields bivalent functional completeness: {and, or, not, non} can express every function f: {T, U, F}^n → {T, F}.
3. `non` enables *detection* of uncertainty rather than mere propagation. Without it, Kleene logic can pass uncertainty through but can never see it.

This is a genuine structural enrichment of Strong Kleene logic.

See also

- BuddhistParallels – pramana theory, tetralemma, and the Buddhist motivation for non-affirming negation.

Training

This document covers the full training infrastructure for WikiOracle's NanoChat integration: how raw data is preprocessed into structured training examples, how DegreeOfTruth governs learning, the dynamic systems interpretation, and the online training pipeline that ties it all together.

DegreeOfTruth (DoT)

DegreeOfTruth is a bipolar scalar on -1 .. $+1$ that measures how well a user's TruthSet agrees with the server's collected truth.

```
DegreeOfTruth = 2 $\times$ mean(agreement_i) $-$ 1    for shared entries
```

where:

```
agreement_i = 1 $-$ |server_trust_i $-$ client_trust_i| / 2
```

The bipolar range encodes both true and false statements:

- **DoT = +1**: the user's claims fully agree with the server – the exchange is true. Train at full learning rate.
- **DoT = −1**: the user's claims fully contradict the server – the exchange is false. Train at full learning rate (learning what is *not* true is as valuable as learning what *is* true).
- **DoT ≈ 0**: no shared entries, or perfect cancellation – nothing to learn. Skip training.

Both poles ($+1$ and -1) train at full strength via $|\text{DoT}|$; only the zero crossing results in a skip. The sign encodes direction (agree/disagree), not magnitude.

This is a placeholder. A future version should incorporate `compute_derived_truth` (operator propagation) before comparison.

Dynamic Systems Perspective

With a bipolar DegreeOfTruth (-1 .. $+1$), the training loop forms a **dynamic equation with both poles and zeros**: DoT $= +1$ and DoT $= -1$ are attracting poles where the system learns at full strength (truths and refuted falsehoods respectively), while DoT $= 0$ is a zero – an equilibrium point where no learning occurs.

This structure resembles a **Hopfield network**, where the energy landscape has stable attractors (memorised patterns) and unstable saddle points. In our system:

- The **TruthSet** plays the role of the weight matrix, encoding the collective memory of what is true and what is false.
- Each **training step** is a state transition that pushes the model toward one of the attractors (truth or refutation).
- The **zero crossing** (DoT ≈ 0) is the energy barrier between attractors – the point of maximum uncertainty where the system has insufficient signal to commit to either direction.

As the server TruthSet accumulates entries from multiple users, the poles and zeros of this dynamic equation shift. The slow-moving average merge ensures that attractors are stable under small perturbations (anti-capture), while strong consensus can still move the landscape over time. This is analogous to the

annealing process in Hopfield networks, where the energy landscape gradually settles into deeper minima as more patterns are stored.

Zero-Mean Balance

The DoT-annealed training algorithm has a **zero-mean balance** property: over many interactions with diverse DoT values, the expected net gradient contribution tends to zero. This arises because:

- DoT = +1 and DoT = −1 both train at full strength but push in opposite directions (learning truth vs. learning falsehood).
- DoT near 0 contributes nearly nothing (the sigmoid zero-crossing).
- Over a diverse population of users, the average DoT tends toward zero if the TruthSet is balanced.

This is directly analogous to the **symmetric energy wells** of Hopfield networks. In a Hopfield network, stored patterns and their complements are both stable attractors. In our system, truths and refuted falsehoods are both stable attractors, and the zero-crossing is the energy barrier between them.

The EMA weight anchoring adds a third stabilizing force: regardless of the DoT distribution, the model is always gently pulled back toward its checkpoint state. This is analogous to the **temperature decay** in simulated annealing – as training progresses, the system becomes increasingly resistant to perturbation while allowing deeper learning within established wells.

Sigmoid Warmup as Annealing Schedule

The sigmoid warmup schedule serves as a **cooling schedule** for the annealing process. When online training is first enabled:

1. The TruthSet is empty or sparse – DoT values are unreliable.
2. The warmup suppresses the learning rate to near-zero.
3. As interactions accumulate, the TruthSet gains signal.
4. The warmup ramps up, allowing the model to learn from now-reliable DoT values.
5. At steady state (step » warmup_steps), the warmup factor is ~1.0 and training proceeds at full configured strength.

This mirrors the annealing schedule in physical systems: high temperature (high exploration, low commitment) → low temperature (low exploration, high commitment to learned patterns).

Sensation – Preprocessing Pipeline

`bin/sensation.py` transforms plain-text conversations into XML-tagged training data so that NanoChat learns WikiOracle's structured protocol. The name follows the epistemological pipeline: **Sensation** → **Perception** → **Cognition** – raw input data (sensation) is structured and tagged before it reaches the model (cognition).

Sensation works for both batch retagging of the NanoChat SFT corpus (`identity_conversations.jsonl`, ~14K conversations) and dynamic training examples from the online training pipeline.

JSONL Record Types

The output JSONL uses four record types, splitting the client identity into separate User and Server records:

Type	Tag	Purpose
`user`	`<User>`	User identity – username, uid, timestamp
`server`	`<Server>`	Server identity – name, version, timestamp
`conversation`	`<Conversation>`	Messages with `<Q>` (query) and `<R>` (response) pairs
`truth`	`<Truth>`	Extracted factual claims with trust and spacetime

Inside message content, user messages are wrapped in `<Q>...</Q>` and assistant messages in `<R>...</R>`. Each sentence within a message is independently classified as `<fact>` or `<feeling>`:

```
<R><feeling>That is a great question!</feeling>
<fact trust="0.5"><place>Paris</place>Paris is the capital of France.</fact>
<feeling>I hope that helps.</feeling></R>
```

Korzybski IS Detection

Alfred Korzybski (*Science and Sanity*, 1933) observed that the English copula "is" conflates several distinct relations:

- **IS of identity**: "Socrates is a man"
- **IS of predication**: "The sky is blue"
- **IS of existence**: "There are eight planets"

Each asserts something verifiable about the world – a *fact* bound to a specific spacetime context. "The cup is on the table" is only true at a particular place and time; at a different spacetime it may not be.

The heuristic classifier in `sensation.py` detects these patterns:

Subtype	Pattern	Example
Identity	"X is/are [a/an/the] Y"	"Socrates is a man"
Predication	"X is/are ADJ"	"The sky is blue"
Existence	"there is/are X"	"There are 8 planets"
Mereological	"X contains/includes Y"	"Water contains hydrogen"
Quantity	"X has/have N Y"	"I have 3 cats"
Definition	"X is called/known as Y"	"A polygon is defined as…"

Sentences without an IS pattern, or with subjective markers ("I think", "maybe", "might be"), questions, or meta-discourse ("That's a great question") default to `<feeling>`. Auto-detected facts receive `trust=0.5` – a conservative default that flags them for future verification. Spatiotemporal binding is expressed via `<place>` and `<time>` child elements inside `<fact>` or `<feeling>` tags.

Usage

Batch-convert a corpus:

```
make preprocess
# or: python bin/sensation.py corpus input.jsonl output.jsonl
```

Classify a single sentence:

```
python bin/sensation.py tag "Paris is the capital of France."
# $\rightarrow$ Classification: fact (identity)
# $\rightarrow$ Tagged: <Q><fact trust="0.5">Paris is the capital of France.</fact></Q>
```

In the online training pipeline, `response.py` calls `preprocess_training_example()` automatically before each `/train` POST, so all training examples are XML-tagged without manual intervention.

Online Training

Purpose

Enable continuous online training of NanoChat where every interaction can trigger a weight update. The learning rate is governed by DegreeOfTruth (see above) which evaluates how well the user's claims fit the collective evidence.

WikiOracle accomplishes this by:

1. Maintaining a **server-owned TruthSet** (`truth.xml`) that accumulates facts from all users.
2. Computing a **DegreeOfTruth** (-1 .. $+1$) per interaction.
3. Using |DegreeOfTruth| to modulate the **learning rate** of a one-step online training pass in NanoChat.

Conversations are **not** stored on the server. Only **knowledge** truth entries are retained. The server TruthSet contains knowledge facts, operators, authorities, and references – no feelings, provider entries, or **news facts** (spatiotemporally bound observations).

News facts (those with `<place>` or `<time>` child elements carrying real values) are session-only. Persisting them would risk "worldline capture" – an observer could reconstruct a user's physical trajectory through spatiotemporal observations. The `filter_knowledge_only()` function in `bin/truth.py` enforces this boundary.

User Identity

Each user is identified by a pseudonymous GUID derived deterministically from `client_name` in the state file (UUID-5 in the WikiOracle namespace). This GUID is stored at the root level of the user's state and used internally when merging truth entries into the server table.

Pipeline

The pipeline is staged so the user gets a response first; truth merging and training happen after the response is delivered.

Stage 1 – Respond

1. Receive the user's query and TruthSet.
2. Use truth entries for RAG as usual.
3. Return the response to the user.

Stage 2 – Compute DegreeOfTruth

4. Score the user's TruthSet against the server's current truth (formula above).

Stage 3 – Update server TruthSet

5. Filter client truth per the Entanglement Policy (doc/Entanglement.md):
   - When `store_concrete` is false (default), only universal facts pass through (`filter_knowledge_only()`).
   - Entries with identifiable content are always filtered regardless of `store_concrete` (`detect_identifiability()`).
   - Feelings never reach the merge (`_is_server_storable()` rejects them).
   - Operators may compose facts or feelings as operands.
   - Feelings are also stripped from training messages (`strip_feelings_from_training()` in `bin/sensation.py`).
6. Merge the surviving truth entries into the server TruthSet (`truth.xml`):
   - **Match found**: nudge the server entry's trust toward the incoming value using a slow-moving average: `server_trust += merge_rate $\times$ (client_trust $-$ server_trust)`
   - **No match**: insert the entry with the stated trust value.
   - Entries are restricted to facts, operators, authorities, and references. Feelings and provider entries are not stored.

Stage 3a – TruthSet Trimming

When the server TruthSet exceeds `truth_max_entries` (default 1000), the merge step trims the table by removing entries with `|trust|` closest to 0.0 (no information value), keeping entries with highest `|trust|` (strongest signal, positive or negative). This is checked during the merge step, not as a separate operation. The trim count is logged.

The `truth_max_entries` parameter is configurable per-user via the Settings dialog (see UserInterface.md) and defaults to 1000 in the chat config.

Stage 4 – Tag and Train

7. Preprocess the training example through Sensation (`sensation.py`) to add `<Q>`/`<R>` and `<fact>`/`<feeling>` XML tags.
8. Strip `<feeling>` blocks from training messages (`strip_feelings_from_training()` in `bin/sensation.py`) – feelings must never train model parameters (Entanglement policy, doc/Entanglement.md).
9. Call NanoChat's online training endpoint (`POST /train`) with the tagged prompt, DegreeOfTruth, and training hyperparameters.

Device Configuration

Online training runs on the device specified by `server.training.device` in the config file (`config.xml`). Valid values:

- `cpu` (default) – safe for the WikiOracle production server
- `cuda` – use NVIDIA GPU if available
- `mps` – Apple Metal Performance Shaders (macOS)
- `auto` – probe CUDA → MPS → CPU and use the best available

The model is moved to the training device for the gradient step, then moved back to the inference device afterward.

Training Algorithm – DoT-Annealed Selective Training

The `/train` endpoint (`bin/nanochat_ext.py`) implements a carefully designed online training algorithm that prevents weight collapse while enabling meaningful learning from each interaction.

Consistent AdamW Optimizer All parameter groups use **AdamW** consistently. The batch training regime uses Muon (Newton-Schulz orthogonalization) for transformer matrices, but online training uses AdamW for all groups because:

1. **No persistent optimizer state** – the optimizer is created fresh each `/train` call. Muon's advantage comes from accumulated momentum; without persistent state, its Newton-Schulz orthogonalization provides limited benefit.
2. **Effectively sign-SGD** – AdamW without momentum history is effectively sign-SGD: simpler, faster, well-understood.
3. **Independent interactions** – each `/train` call is fully independent. A bad gradient cannot poison future steps because there is no momentum history to contaminate. The effective step size is bounded by `lr`.

Parameter Groups The optimizer creates six parameter groups that mirror the production training regime in `nanochat/nanochat/gpt.py:GPT.setup_optimizer()`:

Group	Base LR	Parameters
`lm_head`	0.0027	Output projection weights (language model head)
`wte`	0.136	Token embedding weights
`value_embeds`	0.136	Value residual stream embeddings
`resid_lambdas`	0.005	Per-layer residual scaling scalars
`x0_lambdas`	0.5	Skip-connection blending scalars
`transformer_h`	0.02	All transformer block matrix parameters (attention, MLP, norms)

Parameters not matching any named group are included in `transformer_h`. All groups use the same AdamW optimizer – no Muon.

Learning Rate Modulation The effective learning rate for each parameter group is computed as:

```
lr_effective = lr_base $\times$ lr_scale
```

where:

```
lr_scale = (truth_weight $\times$ |DoT| + (1 - truth_weight)) $\times$ sigmoid_warmup(step)
```

The **truth_weight** parameter (0.0-1.0) controls how much DoT gates the learning rate:

`truth_weight`	`lr_effective`	Behavior
0.0	`lr_base $\times$ warmup`	Vanilla SFT – full LR regardless of DoT. No truth bias.
0.5	'lr_base × (0.5 ×	DoT
1.0	'lr_base ×	DoT

The `truth_weight` replaces the former boolean `rag` checkbox in the Settings dialog. At `truth_weight=0`, the system trains on everything like vanilla SFT (feelings-only, no truth bias). At `truth_weight=1`, DoT governs learning completely.

The RAG flag (`rag`) remains separate conceptually: `truth_weight > 0` controls whether truth entries *influence training*, while truth entries are *sent to the provider for context* whenever `truth_weight > 0`.

Sigmoid Warmup The sigmoid warmup schedule prevents early random updates from corrupting the model when online training is first enabled:

```
sigmoid_warmup(step) = 1 / (1 + exp(-k $\times$ (step - midpoint)))
```

Step	Warmup value	Effect
0	~0.007	Nearly zero – first interactions barely train
midpoint	0.5	Half strength
2 × midpoint	~0.993	Nearly full strength – training ramp is complete

The `warmup_steps` parameter (default 50) is the sigmoid midpoint. The steepness parameter `k` is fixed at 0.1.

This schedule ensures that the first few interactions after enabling online training have minimal weight impact, allowing the TruthSet to accumulate signal before the model commits to learning from it.

Gradient Clipping After the backward pass, gradients are clipped to prevent catastrophic single-step weight changes:

```
torch.nn.utils.clip_grad_norm_(all_params, max_norm=grad_clip)
```

The `grad_clip` parameter (default 1.0) bounds the global gradient norm. The actual gradient norm after clipping is returned in the `/train` response as `grad_norm` for monitoring.

If `grad_norm` consistently hits the clip threshold, this indicates either: (a) the learning rate is too high for the current data, or (b) the training example is an outlier. Both are handled gracefully by clipping – the step still proceeds, just with bounded magnitude.

EMA Weight Anchoring (Anti-Capture) After each optimizer step, model weights are blended back toward the **checkpoint anchor** using exponential moving average:

```
for p, anchor in zip(model.parameters(), anchor_params):
    p.data.lerp_(anchor, anchor_effective)
```

where:

`anchor_effective = anchor_decay $\times$ truth_weight`

The anchor weights are the original checkpoint weights, initialized on the first `/train` call. The `anchor_decay` parameter (default 0.001) controls the blend-back rate.

Key properties:

- **truth_weight=0**: `anchor_effective = 0` – no anchor pull. Weights drift freely (vanilla SFT mode).
- **truth_weight=1**: `anchor_effective = anchor_decay` – full anchor pull. Weights are always tugged back toward the checkpoint.
- The anchor prevents **gradual drift** while allowing meaningful learning. Over many steps, the weights can move significantly from the checkpoint, but each individual step is bounded.
- **Reset via `make nano_restart`** – reloads both the model and the anchor, resetting the training state.

The anchor is stored in `app.state.anchor_params` as a list of cloned parameter tensors. The step count is stored in `app.state.train_step_count`.

Training Flow (Per `/train` Call)

```
POST /train
  |- Clamp DoT to [-1, +1], truth_weight to [0, 1]
  |- If truth_weight > 0 and |DoT| < 1e-6 -> skip (no signal)
  |- Acquire model worker from pool
  |- Tokenize messages (bos + user_start/end + assistant_start/end)
  |- If < 2 tokens -> skip (empty)
  |- Initialize EMA anchor weights (first call only)
  |- Move model to training device
  |- Build 6 parameter groups (fresh each call)
  |- Compute lr_scale = (tw x |DoT| + (1 - tw)) x sigmoid_warmup(step)
  |- Scale each group's LR: lr_base x lr_scale
  |- Create fresh AdamW optimizer
  |- Forward pass -> cross-entropy loss
  |- Backward pass
  |- Gradient clipping -> grad_norm
  |- Optimizer step
  |- EMA anchor blend-back (if truth_weight > 0)
  |- Increment step count
  |- Move model back to inference device
  `- Return {status, loss/gain, step, grad_norm}
```

The response uses key `"loss"` for positive DoT (learning truth) and `"gain"` for negative DoT (learning falsehood). Both are the cross-entropy loss value; the key name indicates the semantic direction.

Configuration Summary

Parameter	Config path	Default	Description
`truth_weight`	`server.truthset.truth_weight`	0.7	DoT gating strength (0=vanilla SFT, 1=full Do
`warmup_steps`	`server.training.warmup_steps`	50	Sigmoid warmup midpoint
`grad_clip`	`server.training.grad_clip`	1.0	Max gradient norm
`anchor_decay`	`server.training.anchor_decay`	0.001	EMA blend-back rate
`truth_max_entries`	`server.training.truth_max_entries`	1000	Max server TruthSet size
`device`	`server.training.device`	`"cpu"`	Training device

See Config.md Section Server for the full configuration reference.

Truth-Modulated Loss (MentalModel) In the basicModel's `MentalModel`, the training loss is further modulated by **luminosity** and **universality** to prevent learning irrational or unkind propositions:

```
totalLoss = totalLoss * (1 + LuminosityWeight * (1 - luminosity)
                          + UniversalityWeight * (1 - universality))
```

- **Luminosity** measures truth set coherence (see Logic.md Section Luminosity). Low luminosity increases loss.
- **Universality** measures the Golden Rule via SVO/OVS swap (see Ethics.md Section Universality). Low universality increases loss.

Both weights default to 0.1, making the effect gentle. This operates independently of the DoT-annealed training described above – DoT gates the learning rate, while luminosity/universality modulate the loss magnitude.

See Config.md Section MentalModel Configuration for the XML parameters.

SFT Corpus Preparation

The `prepare_sft_corpus()` function in `bin/sensation.py` provides batch conversion of the NanoChat SFT corpus to XML-tagged format with feelings stripped. This is the batch equivalent of `preprocess_training_example()` and is used when retraining NanoChat from scratch with WikiOracle's protocol:

```
make build_sft
# or: python bin/sensation.py sft input.jsonl output.jsonl
```

Each input line is a JSON array of `{"role": ..., "content": ...}` message dicts. Output lines are the same shape but with content XML-tagged using `<Q>`/`<R>` and `<fact>`/`<feeling>` tags, with feelings stripped (matching the online training pipeline).

Server TruthSet

The server TruthSet is stored as `truth.xml` in the same XML format used for state files (WikiOracle State). Each stored entry is a typed truth element:

```
<fact id="..." title="..." DoT="0.8" time="..." place="Athens">
  All men are mortal.
</fact>
```

Resolution: Before entries reach the server TruthSet, they are resolved by `resolve_entries()` in `bin/truth.py`:

- `<reference>` → `<fact src="domain">text</fact>` (domain preserved for deeper lookup)
- `<authority>` → list of `<fact src="domain">content</fact>` (fetched from remote, trust scaled)
- `<provider>` → `<feeling>` (provider responses are treated as feelings until providers can report truth claims with DoT)
- `<fact>`, `<feeling>`, `<logic>` → pass through unchanged

Entry types stored after resolution: `<fact>` (knowledge – no `<place>`/`<time>` children) and `<logic>` entries (operators wrapped in `<logic><and|or|not|non>...</logic>`).

Entry types **not** stored: `<feeling>`, `<provider>`, unresolved `<reference>`, unresolved `<authority>`, and (when `store_concrete` is false) news facts with `<place>` or `<time>` child elements. Content matching identifiability patterns (PII) is always excluded.

News facts are spatiotemporally bound – persisting them risks worldline capture. Whether to store them is a user choice controlled by `store_concrete` in config.xml (default false), consistent with Zero-Knowledge / Selective Disclosure principles. See `filter_knowledge_only()` in `bin/truth.py` and `detect_identifiability()` for PII detection. See doc/Entanglement.md for the full policy.

The server TruthSet includes the user GUID as a trust entry, so the server can track per-user trust alongside factual claims.

Anti-Capture

The online training system has multiple layers of defense against weight collapse and capture by any single user:

TruthSet level:

- Entries are merged with a **slow-moving average** (`merge_rate`, default 0.1), so no single user can instantly override collective truth.
- Disproven entries naturally drift toward -1 as contradicting evidence accumulates from other users.
- **TruthSet trimming** (`truth_max_entries`, default 1000) removes low-signal entries ($|\text{trust}|$ near 0), keeping the TruthSet focused on strong signal.

Training level:

- **DegreeOfTruth gates the learning rate** – claims that diverge from consensus (DoT near 0) have minimal training impact.
- **Gradient clipping** (`grad_clip`, default 1.0) prevents any single training step from making catastrophic weight changes.
- **EMA weight anchoring** (`anchor_decay`, default 0.001) continuously blends weights back toward the checkpoint, preventing gradual drift. The anchor pull is scaled by `truth_weight`, so at full truth-gating the model is always tugged back toward its initialized state.
- **Sigmoid warmup** (`warmup_steps`, default 50) prevents the first few interactions from corrupting weights before the TruthSet has accumulated sufficient signal.
- **Fresh optimizer per call** – no persistent momentum state means a bad gradient cannot poison future steps. Each interaction is independent.
- **Consistent AdamW** – without accumulated Muon state, AdamW is effectively sign-SGD with bounded step size.

Operational level:

Manual rollback is available via Makefile targets:

- `make checkpoint-pull` – rsync SFT checkpoints from the remote WikiOracle server to `output/checkpoints/` for safekeeping.
- `make checkpoint-push` – restore checkpoints from backup, then `make wo-restart` to reload weights.

The intended workflow: pull a checkpoint before enabling online training, then push to restore if capture is detected.

Dissonance Detection and Pluralistic Truth (TODO)

Detecting and resolving dissonance within the server TruthSet is left for future work. The goal is to support a higher-dimensional truth-space where contradictory claims can coexist when they originate from different perspectives.

For example: "the world was created in seven days" and "the world was created over millions of years" could both be maintained as true from their respective perspectives. This requires embedding perspective alongside truth value so that the TruthSet becomes a manifold rather than a flat list.

In the current consensus model, a DoT ≈ 0 simply means "nothing to learn" and the training step is skipped. In a future **pluralistic** model – where the same claim can be true in context c_1 and false in context c_2 – a DoT of 0 may instead indicate that **user feedback is needed** to disambiguate which context applies before training should proceed.

Possible approaches include context/perspective tags on entries, truth-space embeddings with frame clustering, conditional truth values indexed by worldview, or explicit user prompts to resolve ambiguity

when the TruthSet produces conflicting signals.

OpenClaw Integration

OpenClaw is WikiOracle's multi-channel front-end (Slack, Discord, Telegram, etc.). The TypeScript extension at `openclaw/extensions/wikioracle/` registers WikiOracle as a native OpenClaw provider by spawning `bin/wo` for each query.

The extension provides three capabilities:

1. **Provider** – WikiOracle appears in OpenClaw's provider selector alongside OpenAI, Anthropic, etc. Selecting it routes all messages through the WikiOracle server's full pipeline.
2. **Command** (`/wo <message>`) – Direct CLI-style access from any OpenClaw channel, bypassing the agent/LLM layer.
3. **Tool** (`wikioracle_query`) – Lets OpenClaw agents invoke WikiOracle programmatically during agentic runs.

The message flow through the training pipeline:

1. OpenClaw receives a message from any channel (Slack/Discord/etc.).
2. The wikioracle extension spawns `bin/wo` with the message.
3. `bin/wo` sends `POST /chat` to the WikiOracle server.
4. WikiOracle responds (Stage 1: provider routing, truth RAG).
5. Stages 2-4 execute server-side: DoT computation, truth merge, Sensation preprocessing, and NanoChat online training.
6. `bin/wo` prints the response to stdout; the extension captures it and relays it back to the originating channel.

In stateful mode (default), the WikiOracle server owns session state – conversation context persists across messages without client-side storage. In stateless mode, state is serialized to a local XML file via `bin/wo -f`.

The training pipeline treats OpenClaw messages identically to direct HTTP or web UI clients – the same DoT computation, TruthSet merge, Sensation preprocessing, and `/train` call apply. The `bin/wo` CLI is transparent to the training system.

Extension configuration

```
// In OpenClaw's config file (~/.openclaw/config.json5 or project-level)
{
  plugins: {
    entries: ["wikioracle"],
    wikioracle: {
      woPath: "/path/to/WikiOracle/bin/wo",
      serverUrl: "https://127.0.0.1:8888",
      insecure: true,    // skip TLS verification for local dev
      stateful: true,    // server owns session state
      token: "...",      // optional bearer token
    },
  },
}
```

See `openclaw/extensions/wikioracle/openclaw.plugin.json` for the full config schema.

Server TruthSet Visibility (Debug Mode)

When `DEBUG_MODE` is enabled and online training is active, the `/chat` response includes a `server_truth` key containing the server's TruthSet entries formatted as authority-tagged entries:

```
{
  "server_truth": [
    {
      "type": "authority",
      "id": "entry-id",
      "title": "...",
      "trust": 0.8,
      "content": "<fact>...</fact>",
      "source": "wikioracle"
    }
  ]
}
```

The client displays these entries in the TruthSet with a visual marker (server badge, different border style). Entries are deduplicated by `id` on each debug refresh.

The `server_id` field in `config.xml` under `<server>` provides a stable identifier for this server instance (default: `"wikioracle"`).

Implementation

System overview

WikiOracle is a local-first Flask shim that sits between a browser UI and one or more upstream LLM providers. Conversations are stored as a hierarchical tree – each conversation is an ordered list of messages that may branch into child conversations.

```
Browser  --HTTP-->  wikioracle.py  --HTTP-->  Upstream LLM
                        |
                    state.xml
                  (persistent state)
```

Components

Layer	File(s)	Role
Server	`bin/wikioracle.py`	Flask app – `/chat`, `/state`, `/merge`
Config	`bin/config.py`	Config dataclass, XML loader, provi
State library	`bin/state.py`	Pure-Python tree operations, XML
Response	`bin/response.py`	Chat pipeline, provider coordination
Truth	`bin/truth.py`	Trust processing, authority resolutio
Sensation	`bin/sensation.py`	Preprocessing: Korzybski IS detecti
OpenClaw	`openclaw/` (git submodule) + `openclaw/extensions/wikioracle/`	Multi-channel front-end (Slack/Disc
NanoChat ext	`bin/nanochat_ext.py`	`POST /train` route mounted onto N
Client app	`client/wikioracle.js`	State management, API calls, messa
Client config	`client/config.js`	Config global, sessionStorage persist
Client state	`client/state.js`	State global, sessionStorage persiste
Client utils	`client/util.js`	Shared helpers, settings panel, confi
Client query	`client/query.js`	Server communication layer, convers
Tree renderer	`client/tree.js`	D3.js top-down hierarchy – layout, r
Shell	`client/index.html`	Single-page app: layout, CSS, settin
State schema	`data/state.xsd`	XSD schema for XML state files (W
Config schema	`data/config.xsd`	XSD schema for `config.xml` validat
Tests	`test/test_*.py`	Tests covering state, stateless contra

Server endpoints

Method	Path	Purpose
GET	`/health`	Liveness check
GET	`/server_info`	Stateless flag + url_prefix
GET	`/bootstrap`	One-shot seed for stateless clients (state + config)
GET	`/info`	State/schema/provider metadata for diagnostics
GET	`/state`	Return current in-memory state
POST	`/state`	Replace state
GET	`/state_size`	State file size in bytes (progress bar)
POST	`/chat`	Send a message – append to existing conversation, branch, or create new root
POST	`/merge`	Merge an imported state file into current state
GET	`/config`	Normalized config (includes provider metadata and defaults)
POST	`/config`	Accept full config dict; write config.xml to disk
GET	`/`	Serve `client/index.html`
GET	`/<file>`	Serve static assets from `client/`

Data flow: client <-> server

Truth, context, and output are **client-owned**. They flow client → server only; the server never sends them back in a chat response.

```
POST /chat request:   client sends truth + context + output + message
POST /chat response:  server returns text + conversation delta
                      (no truth, no context, no output)
```

In stateful mode, the server persists the full state (including the client-supplied truth) to `state.xml`. On error rollback, the client reloads conversations from the server but preserves its own truth, context, and output.

In stateless mode, the server has no disk – the client sends and receives the full state.

Chat routing

`POST /chat` accepts:

- `conversation_id` – append user message + LLM response to an existing conversation
- `branch_from` – create a new child conversation under the specified parent, seed it with the user message + LLM response
- Neither – create a new root-level conversation

Thought-Free Mode

When `thought_free` is set in the query config, the chat pipeline applies two constraints:

1. **LLM providers:** `build_query()` prepends a 10-rule constraint prompt to `bundle.system` that enforces one-pointed, non-discursive output (no epistemic verbs, no observer, no comparisons, sentences $\leq$ 10 words).
2. **BasicModel provider:** `_call_basicmodel()` sends `"thought_free": true` in the JSON payload. `serve.py` sets `TheGrammar.thought_free = True` before inference (restricting S-tier to the S→C transition only) and resets it in a `finally` block.

The flag flows: `wikioracle.js` query config → `POST /chat` → `process_chat()` → `_call_provider()` → `build_query()` / `_call_basicmodel()`.

Reducing Inferential Truth to Facts and Feelings

The chat pipeline always sends the final query to the provider selected in the UI config. The `rag` flag decides whether `state.truth` participates at all: when `rag` is false, no truth entries are sent; when `rag` is true, the server reduces inferential structure to propositional entries plus derived certainty before the final synthesis step. When dynamic `<provider>` entries exist, the UI-selected provider acts as the alpha and the ranked provider entries act as beta experts.

```
st = static_truth(state.truth)      # facts, feelings, references
t  = st + dynamic_truth(st)         # operators, authorities, providers
```

Structural entries such as `<provider>`, `<operator>`, and `<authority>` are not treated as propositions; they are evaluated against the static set and contribute their results back to the final bundle.

When a user sends a query:

1. **Static truth**: `static_truth()` extracts the evaluable subset of `state.truth`: facts, feelings, and references.
2. **Dynamic truth**: `dynamic_truth(st)` evaluates structural entries against that subset.
   - **Operators**: `compute_derived_truth()` evaluates `<and>`, `<or>`, and `<not>` with Strong Kleene semantics and propagates derived certainty.
   - **Authorities**: `<authority>` entries fetch remote TruthSets and append their entries with scaled certainty.
   - **Providers**: ranked `<provider>` entries are queried as beta experts. Each beta receives the query bundle, conversation history, output instructions, and optionally the alpha's preliminary response as a steering signal.
3. **Final synthesis**: the alpha receives all original `state.truth` entries, operator-derived certainty, authority entries, beta responses, conversation history, system context, and the user message.
4. **Single-call case**: when there are no dynamic `<provider>` entries, the flow collapses to one call to the UI-selected provider.

This is the HME (Hierarchical Mixture of Experts) model: operators compute derived certainty over the static truth, authorities contribute remote knowledge, and dynamic providers act as expert consultants while the UI-selected provider synthesizes the final answer. Provider credentials remain separate: each provider uses its own key, whether configured in `config.xml` or embedded in the truth entry. See Voting.md for the theoretical foundation.

State library (`bin/state.py`)

Key functions:

Function	Purpose
`state_to_xml(state)`	Serialise nested tree → XML string
`xml_to_state(text)`	Parse XML → nested tree
`atomic_write_xml(path, state)`	Atomic XML file write (temp + fsync + rename)
`load_state_file(path)`	Load state from XML file
`find_conversation(convs, id)`	Recursive tree lookup
`get_ancestor_chain(convs, id)`	Walk up to root, return list of ancestors
`get_context_messages(convs, id)`	Ancestor chain messages in order (for LLM context)
`add_message_to_conversation(convs, id, msg)`	Append a message
`add_child_conversation(convs, parent_id, child)`	Insert a new branch
`remove_conversation(convs, id)`	Delete a subtree
`ensure_minimal_state(state)`	Fill in missing fields with defaults

Config

WikiOracle configuration lives in `config.xml` at the project root. The file is validated by `data/config.xsd` and loaded at server startup by `bin/config.py`. A template with sensible defaults ships as `data/config.xml` – copy it to the project root and fill in your values. `config.xml` is gitignored.

The configuration has two top-level sections: Server and Providers. Each section is described below with its fields, defaults, and design rationale.

> **Migration note (v0.x):** The former `user`, `chat`, and `ui` top-level sections have been removed. User identity (`client_name`, `client_id`) is now in State. Chat evaluation fields moved to `server.evaluation`; truth policy fields moved to `server.truthset`; UI preferences moved to `state.client.ui`. See each section below for the full mapping.

Server

Runtime parameters. These values serve as defaults and are typically overridden via CLI flags at startup.

Field	Type	Default	Description
`server_name`	string	`"WikiOracle"`	Human-readable display name for this server instance.
`server_id`	string	`"wikioracle"`	Stable identifier for this server instance. Used as the `source` field in server trut
`stateless`	boolean	`false`	Stateless mode – no disk writes. Equivalent to `--stateless` CLI flag. See Enta
`url_prefix`	string	`""`	URL path prefix prepended to all routes (e.g. `/chat`) for reverse-proxy deploym
`truthset`	section	–	TruthSet policy settings. See Truthset below.
`evaluation`	section	–	LLM inference defaults. See Evaluation below.
`training`	section	–	Online learning subsystem. See Training below.
`allowed_urls`	section	–	URL whitelist for authority/provider fetches. See Allowed URLs below.

```
<server>
  <server_name>WikiOracle</server_name>
  <server_id>wikioracle</server_id>
  <stateless>true</stateless>
  <url_prefix></url_prefix>

  <truthset>...</truthset>
  <evaluation>...</evaluation>
  <training>...</training>
  <allowed_urls>...</allowed_urls>
</server>
```

Truthset

TruthSet policy settings controlling how facts are stored and validated.

Field	Type	Default	Description
`truth_symmetry`	boolean	`true`	Enforce Truth Symmetry. Claims involving value judgements are checked for
`store_concrete`	boolean	`false`	Store concrete (spatiotemporally-bound) facts in the server TruthSet. When
`truth_weight`	decimal 0.0-1.0	`0.7`	Controls how much DegreeOfTruth (DoT) gates the online training learning

```
<truthset>
  <truth_symmetry>true</truth_symmetry>
  <store_concrete>false</store_concrete>
</truthset>
```

Truth weight and RAG The `truth_weight` field (0.0-1.0) replaces the former boolean `rag` flag. It serves a dual purpose:

1. **RAG delivery gate**: When `truth_weight > 0`, the full TruthSet – facts, feelings, references, operators, authorities, and providers – is assembled into the provider bundle and sent to the UI-selected provider. When `truth_weight = 0`, no truth is sent (equivalent to the former `rag: false`).
2. **Training LR modulation**: During online training, `truth_weight` controls how much DoT gates the learning rate. See Training.md Section Training Algorithm for the formula.

Legacy migration: The former `rag` boolean is automatically migrated: `rag: true` → `truth_weight: 0.7`, `rag: false` → `truth_weight: 0.0`. This migration runs in both the client (`client/config.js`) and server (`bin/response.py`).

Evaluation

Defaults for LLM inference. These were formerly in the top-level `chat` section.

Field	Type	Default	Description
`temperature`	decimal 0.0-2.0	`0.7`	Sampling temperature. 0.0 is deterministic; 2.0 is maximum randomness.
`max_tokens`	positive int	`128`	Maximum tokens requested from the provider for a single response.
`timeout`	positive int	`120`	Request timeout in seconds for provider evaluation.
`url_fetch`	boolean	`false`	Allow the assistant to fetch and incorporate content from URLs referenced in t
`thought_free`	boolean	`false`	Enable thought-free (shamatha speech) mode. Restricts BasicModel grammar

```
<evaluation>
  <temperature>0.7</temperature>
  <max_tokens>128</max_tokens>
  <timeout>120</timeout>
  <url_fetch>false</url_fetch>
</evaluation>
```

Training

Controls the continuous learning pipeline (Stages 2-4). Renamed from the former `online_training` section.
See Training.md for the full design.

Field	Type	Default	Description
`enabled`	boolean	`false`	**Master switch.** When false, the entire post-resp
`truth_corpus_path`	string	`"data/truth.xml"`	Filesystem path to the server TruthSet (XML). I
`truth_max_entries`	positive int	`1000`	Maximum server TruthSet entries before trimmin
`alpha_base`	decimal	`0.01`	Base learning rate for online training weight upd
`alpha_min`	decimal	`0.001`	Minimum learning rate floor. The adaptive sched
`alpha_max`	decimal	`0.1`	Maximum learning rate ceiling. The adaptive sch
`merge_rate`	decimal	`0.1`	Slow-moving exponential average rate for mergin
`device`	`auto` | `cpu` | `cuda`	`"cpu"`	Compute device for training operations. `auto` sel
`dissonance_enabled`	boolean	`true`	Enable cognitive dissonance detection. The train
`operators_dynamic_enabled`	boolean	`true`	Load custom operators. Custom operators exten
`warmup_steps`	positive int	`50`	Sigmoid warmup midpoint for the annealing sche
`grad_clip`	decimal > 0	`1.0`	Maximum gradient norm for `clip_grad_norm_(`
`anchor_decay`	decimal 0.0-1.0	`0.001`	EMA blend-back rate toward checkpoint weights

```
<training>
  <enabled>false</enabled>
```

```
  <truth_corpus_path>data/truth.xml</truth_corpus_path>
  <truth_max_entries>1000</truth_max_entries>
  <alpha_base>0.01</alpha_base>
  <alpha_min>0.001</alpha_min>
  <alpha_max>0.1</alpha_max>
  <merge_rate>0.1</merge_rate>
  <device>cpu</device>
  <dissonance_enabled>true</dissonance_enabled>
  <operators_dynamic_enabled>true</operators_dynamic_enabled>
  <warmup_steps>50</warmup_steps>
  <grad_clip>1.0</grad_clip>
  <anchor_decay>0.001</anchor_decay>
</training>
```

Behavior when training is disabled When `enabled` is `false` (the default), the engine effectively treats all incoming facts as feelings. The Sensation preprocessor (`bin/sensation.py`) still classifies sentences into `<fact>` and `<feeling>` tags at the message level, but the entire post-response pipeline is skipped:

- **No DegreeOfTruth** is computed.
- **No truth merge** occurs – facts from conversation are never promoted into the TruthSet.
- **No PII filtering** or **symmetry checking** runs (there is nothing to filter).
- **No NanoChat training step** is dispatched.

In this mode, facts carry no more lasting weight than feelings. They are tagged in the XML for display purposes, but they are ephemeral – they do not accumulate, persist, or influence future responses via RAG. This is the safe default: the system functions as a stateless chat proxy until the operator explicitly enables truth acquisition.

Allowed URLs

URL prefixes permitted for outbound HTTP(S) requests made by the server during authority lookups and dynamic provider fetches. Only URLs whose prefix matches one of these entries are allowed. `file://` URLs are always blocked unless explicitly whitelisted.

```
<allowed_urls>
  <url>https://api.openai.com/</url>
  <url>https://api.anthropic.com/</url>
  <url>https://generativelanguage.googleapis.com/</url>
  <url>https://api.x.ai/</url>
  <url>https://en.wikipedia.org/</url>
  <url>https://wikiOracle.org/</url>
  <url>https://127.0.0.1:</url>
  <url>https://localhost:</url>
  <url>http://127.0.0.1:</url>
</allowed_urls>
```

See PrivacyAndSecurity.md for the security rationale.

Providers

LLM provider definitions. Each `<provider name="key">` block configures an upstream API endpoint. The `name` attribute is the internal lookup key; `display_name` is the label shown in the chat UI on assistant messages.

Field	Type	Default	Description
`default`	string	`"wikioracle"`	Provider key selected on startup. Must match a `name` in a `<provide`
`context`	XHTML	(empty)	Persistent context block prepended to every query. Moved from stat
`output`	string	(empty)	Output-format instruction block appended to the system prompt. M
`truth_context`	string	(empty)	Default truth context applied to all providers. Optional.
`conversation_context`	string	(empty)	Default conversation context applied to all providers. Optional.

Each `<provider name="key">` block supports:

Field	Type	Default	Description
`display_name`	string	–	Human-readable label tagging assistant messages (e.g. `chatGPT`, `claude`, `gemin`
`username`	string	–	API login / email associated with the provider account.
`url`	URI	(built-in)	API endpoint URL. Built-in defaults exist for known providers.
`api_key`	string	–	API key. Prefer environment variables (`OPENAI_API_KEY`, `ANTHROPIC_API_KEY`,
`default_model`	string	–	Model identifier used when no model is explicitly selected (e.g. `gpt-4o`, `claude`
`timeout`	positive int	`120`	Request timeout in seconds.
`streaming`	boolean	`false`	Use Server-Sent Events (SSE) for streamed responses.

Built-in providers

Key	Built-in name	Built-in default model	API key env var	Model env var
`wikioracle`	`WikiOracle`	(none; template commonly sets `nanochat`)	–	–
`openai`	`OpenAI`	`gpt-4o`	`OPENAI_API_KEY`	`OPENAI_MODEL`
`anthropic`	`Anthropic`	`claude-sonnet-4-6`	`ANTHROPIC_API_KEY`	`ANTHROPIC_MODEL`
`gemini`	`Gemini`	`gemini-2.5-flash`	`GEMINI_API_KEY`	`GEMINI_MODEL`
`grok`	`Grok`	`grok-3-mini`	`XAI_API_KEY`	`XAI_MODEL`
`openrouter`	`OpenRouter`	`google/gemma-3-4b-it:free`	`OPENROUTER_API_KEY`	`OPENROUTER_MODEL`

Custom providers can be added by appending `<provider name="my_key">` blocks. Any `name` not in the built-in list creates a new provider entry.

```
<providers>
  <default>wikioracle</default>

  <provider name="wikioracle">
    <display_name>oracle</display_name>
    <username>you@example.com</username>
    <url>http://127.0.0.1:8000/chat/completions</url>
    <api_key></api_key>
    <default_model>nanochat</default_model>
    <timeout>120</timeout>
    <streaming>true</streaming>
  </provider>

  <!-- additional provider blocks ... -->
</providers>
```

API key precedence

1. **Environment variable** (recommended for anything beyond localhost).

2. **config.xml api_key** field – convenient for local dev, but the config is served to the client via /bootstrap and /config.
3. **Truth entry** <provider><api_key>$ENV_VAR</api_key></provider> – the $ prefix triggers server-side env-var resolution; the literal key is never stored in state.

The /config and /bootstrap endpoints expose only has_key (boolean) – never the key itself. See PrivacyAndSecurity.md for details.

Runtime precedence

The effective runtime value for a provider setting may come from the request, config.xml, or environment variables.

Setting	Resolution order
Main provider	POST /chat config.provider → providers.default → built-in wikioracle
Model	POST /chat config.model → provider-specific *_MODEL env var → providers.<name>.
API key	provider-specific *_API_KEY env var → providers.<name>.api_key → missing
Local WikiOracle upstream URL	providers.wikioracle.url → WIKIORACLE_BASE_URL + WIKIORACLE_API_PATH

Environment variables

Runtime configuration can also be set via environment variables. These override config.xml values where applicable.

Variable	Default	Purpose
WIKIORACLE_STATE_FILE	state.xml	Path to local state file (WikiOracle State XML).
WIKIORACLE_BASE_URL	http://127.0.0.1:8000	Upstream NanoChat-compatible base URL.
WIKIORACLE_API_PATH	/chat/completions	Upstream chat endpoint path appended to base U
WIKIORACLE_BIND_HOST	127.0.0.1	Host to bind the Flask server to.
WIKIORACLE_BIND_PORT	8888	Port for the Flask server.
WIKIORACLE_SSL_CERT	~/.ssl/<hostname>.pem	TLS certificate path.
WIKIORACLE_SSL_KEY	~/.ssl/<hostname>-key.pem	TLS private key path.
WIKIORACLE_TIMEOUT_S	120	Network timeout (seconds) for provider requests.
WIKIORACLE_MAX_STATE_BYTES	20000000	Hard upper bound for serialized state size.
WIKIORACLE_MAX_CONTEXT_CHARS	40000	Context rewrite cap for merge appendix generatio
WIKIORACLE_REJECT_SYMLINKS	true	Refuse symlinked state files.
WIKIORACLE_AUTO_MERGE_ON_START	true	Auto-import llm_* files at startup.
WIKIORACLE_AUTO_CONTEXT_REWRITE	false	Enable delta-based context append during merges
WIKIORACLE_MERGED_SUFFIX	.merged	Suffix applied to files after successful import.
WIKIORACLE_ALLOWED_ORIGINS	https://127.0.0.1:8888,...	Comma-separated CORS allowed origins.
WIKIORACLE_API_TOKEN	(empty)	Bearer token for endpoint auth (empty = no auth
WIKIORACLE_STATELESS	(unset)	Set truthy to disable all writes and use in-memory
WIKIORACLE_URL_PREFIX	(unset)	Optional reverse-proxy path prefix.
OPENAI_API_KEY	–	OpenAI API key.
ANTHROPIC_API_KEY	–	Anthropic API key.
GEMINI_API_KEY	–	Google Gemini API key.
XAI_API_KEY	–	xAI (Grok) API key.
OPENROUTER_API_KEY	–	OpenRouter API key.

Provider-specific environment overrides

These environment variables override provider-level defaults without editing config.xml.

Variable	Provider	Purpose	Built-in fallback
OPENAI_MODEL	openai	Override providers.openai.default_model	gpt-4o
ANTHROPIC_MODEL	anthropic	Override providers.anthropic.default_model	claude-sonnet-4-6
GEMINI_MODEL	gemini	Override providers.gemini.default_model	gemini-2.5-flash
XAI_MODEL	grok	Override providers.grok.default_model	grok-3-mini
OPENROUTER_MODEL	openrouter	Override providers.openrouter.default_model	google/gemma-3-4b-it:free

OpenClaw Plugin Config

When using WikiOracle as an OpenClaw provider, the TypeScript extension at `openclaw/extensions/wikioracle/` is configured via OpenClaw's plugin config system. Add the following to your OpenClaw config file (`~/.openclaw/config.json5` or project-level):

```
{
  plugins: {
    entries: ["wikioracle"],
    wikioracle: {
      woPath: "/absolute/path/to/WikiOracle/bin/wo",
      serverUrl: "https://127.0.0.1:8888",
      insecure: true,
      stateful: true,
      stateFile: "state.xml",
      token: "optional-bearer-token",
    },
  },
}
```

Field	Type	Default	Description
woPath	string	"../bin/wo"	Absolute or relative path to WikiOracle's bin/wo CLI. Relative path
serverUrl	string	"https://127.0.0.1:8888"	WikiOracle server URL.
insecure	boolean	true	Skip TLS certificate verification (bin/wo -k). Set to false for prod
stateful	boolean	true	Use stateful mode (server owns session state). When false, state is
stateFile	string	"state.xml"	Local state file path for stateless mode (bin/wo -f). Ignored in stat
token	string	(none)	Optional bearer token for WikiOracle API authentication (bin/wo -

The full JSON Schema for the plugin config is defined in `openclaw/extensions/wikioracle/openclaw.plugin.json`.

The extension registers three capabilities:

1. **Provider** (`wikioracle`) – selectable in OpenClaw's provider list
2. **Command** (`/wo <message>`) – direct CLI access from any channel
3. **Tool** (`wikioracle_query`) – available to OpenClaw agents

See Training.md Section OpenClaw Integration for the message flow and training pipeline details.

CLI flags

```
python bin/wikioracle.py [--config PATH] [--debug] [--stateless] [--no-ssl] [--url-prefix PREFIX] [serve
```

Flag	Description
--config PATH	Path to config.xml (default: project root).
--debug	Enable verbose debug logging.
--stateless	Run in stateless mode (no disk writes).

Flag	Description
`--no-ssl`	Serve over plain HTTP (skip TLS).
`--url-prefix PREFIX`	URL path prefix (e.g. `/chat`) for reverse-proxy deployments.
`serve`	Run the Flask shim server (default).
`merge FILE...`	Merge incoming state files into the current state.

Schema

The XML schema is defined in `data/config.xsd`. Use it for IDE validation and autocompletion:

```
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="config.xsd">
  ...
</config>
```

State

WikiOracle conversation state is persisted as XML, validated by `data/state.xsd`, and managed by `bin/state.py`. The canonical file is `state.xml` at the project root.

Structure

A state file has three top-level parts in order:

```
State $\rightarrow$ Client + Conversation* + Truth?
```

```
<?xml version="1.0" encoding="UTF-8"?>
<state>
  <client>...</client>
  <conversation>...</conversation>
  <conversation>...</conversation>
  <truth>...</truth>
</state>
```

`client` is required. Top-level `conversation` elements may repeat. `truth` is optional.

Default state format

The state file is an XML document containing a `<truth>` section. It may be a full WikiOracle State file (with `<state>` root) or an abbreviated file with just a `<truth>` root:

```
<?xml version="1.0" encoding="UTF-8"?>
<truth>
  <fact id="remote_01" title="Water is wet" DoT="1.0" time="2026-02-27T00:00:01Z">
    Water is wet.
  </fact>
  <fact id="remote_02" title="Fire is hot" DoT="0.9" time="2026-02-27T00:00:02Z">
    Fire is hot.
  </fact>
</truth>
```

Server State

- The server persists only the TruthSet of the State, not any conversations.

- The server is stateless with respect to conversations, which means that state is always passed from (and owned by) the client.
- State files are read from `file://` within the data directory.

Security considerations

- **URL scheme restriction**: Only `https://` and `file://` (within `ALLOWED_DATA_DIR`) are permitted
- **Max response size**: Fetched data is capped at 1 MB
- **Max entries per authority**: At most 1000 trust entries are imported per authority
- **No recursive authorities**: If a remote state file contains `<authority>` entries, they are skipped. There is no transitive fetch chain – only one level of authority delegation is supported.
- **Rate limiting**: The in-memory cache prevents excessive re-fetching within the refresh interval

Client

Client metadata. All fields except `client_name`, `client_id`, and `ui` are required.

Field	Type	Description
`version`	positive integer	State grammar version (currently `2`).
`schema`	string	Schema URL, normally `https://raw.githubusercontent.com/arborrhythms/`
`time_creation`	string (ISO 8601)	Timestamp of initial state creation.
`time_lastModified`	string (ISO 8601)	Timestamp of last state write.
`title`	string	Document or project title.
`client_name`	string (optional)	Display name of the client user.
`client_id`	string (optional)	Persistent client identifier (UUID).
`ui`	element (optional)	UI preferences block (see below).

UI preferences

The `<ui>` block inside `<client>` stores client-owned UI preferences.

Field	Type	Description
`layout`	string	Layout mode (e.g. `horizontal`).
`theme`	string	Color theme (`system`, `light`, `dark`).
`model`	string	Preferred model identifier (may be empty).
`divider_pos`	integer	Divider position as a percentage (`0`–`100`).
`swipe_nav_horizontal`	boolean	Enable horizontal swipe navigation.
`swipe_nav_vertical`	boolean	Enable vertical swipe navigation.
`confirm_actions`	boolean	Whether to prompt before destructive actions.

```
<client>
  <version>2</version>
  <schema>https://raw.githubusercontent.com/arborrhythms/WikiOracle/main/data/state.xsd</schema>
  <time_creation>2026-03-07T12:00:00Z</time_creation>
  <time_lastModified>2026-03-07T12:00:00Z</time_lastModified>
  <title>My Project</title>
  <client_name>Alice</client_name>
  <client_id>a1b2c3d4-...</client_id>
  <ui>
    <layout>horizontal</layout>
    <theme>system</theme>
    <model></model>
```

```
    <divider_pos>0</divider_pos>
    <swipe_nav_horizontal>true</swipe_nav_horizontal>
    <swipe_nav_vertical>false</swipe_nav_vertical>
    <confirm_actions>false</confirm_actions>
  </ui>
</client>
```

Context and output format instructions have moved to `config.providers`. See Config.md.

Truth is client-owned. The server persists it in stateful mode but does not originate it.

Conversations

Conversations form a recursive tree. In XML, each `<conversation>` contains a required `<title>` followed by direct child `<message>` and `<conversation>` elements. In memory, `bin/state.py` still normalizes these into `messages[]` and `children[]`.

Grammar

```
Conversation $\rightarrow$ title + (Message | Conversation)*
Message      $\rightarrow$ <message id="" role="" username="" time=""><content>...</content></message>
```

Conversation attributes

Attribute	Required	Description
`id`	yes	Unique conversation ID.
`parentId`	no	Parent conversation ID, or a comma-separated list of parent IDs for a merge/diamond node. Absen
`selected`	no	Boolean. Selected conversations in the file must form one root-to-node path.

Message attributes

Attribute	Required	Values	Description
`id`	yes	string	Unique message identifier.
`role`	yes	`user` | `assistant` | `system`	Message author role.
`username`	yes	string	Display name.
`time`	yes	ISO 8601	Message timestamp.
`selected`	no	boolean	Optional singleton message selection across the whole state file.

Example

```
<conversation id="c_root" selected="true">
  <title>Animals</title>
  <message id="m1" role="user" username="Alice" time="2026-03-06T10:00:00Z">
    <content><Q><fact trust="0.5">Dogs are mammals.</fact></Q></content>
  </message>
  <message id="m2" role="assistant" username="claude" time="2026-03-06T10:00:05Z">
    <content><R><fact trust="0.9">Yes, dogs are mammals of the family Canidae.</fact></R></content>
  </message>
  <conversation id="c_dogs" parentId="c_root" selected="true">
    <title>Dog breeds</title>
    <message id="m3" role="user" username="Alice" time="2026-03-06T10:01:00Z" selected="true">
```

```
      <content><Q><fact trust="0.5">What breeds are common?</fact></Q></content>
    </message>
  </conversation>
</conversation>
```

The XML file persists selection through `selected="true"` attributes on conversations and messages. `bin/state.py` enforces that selected conversations form a single ancestor path and that selected messages are a singleton. For runtime compatibility, the loader still derives internal helper fields like `selected_conversation` and `selected_message` from those XML flags.

Tree operations

`bin/state.py` exposes:

- `find_conversation(convs, id)`
- `get_ancestor_chain(convs, id)`
- `get_context_messages(convs, id)`
- `add_message_to_conversation(convs, id, msg)`
- `add_child_conversation(convs, parent_id, child)`
- `remove_conversation(convs, id)`

When a message is sent, `get_context_messages()` walks the ancestor chain from the active conversation to the root and concatenates each conversation's `messages[]` list in order.

TruthSet

`truth` is an optional container whose children are typed truth elements. The element name is the truth kind, and metadata lives on the element itself.

Shared truth attributes

Attribute	Applies to	Description
`id`	all truth kinds	Unique truth ID.
`title`	all truth kinds	Human-readable label.
`time`	all truth kinds	Timestamp.
`place`	all truth kinds	Optional place label for envelope-level location metadata.
`DoT`	all truth kinds except `feeling`	Degree of Truth on [-1, +1].

Truth kinds

Element	Meaning
`<fact>`	Knowledge claim or observation.
`<feeling>`	Subjective, non-truth-evaluable statement. No `DoT` attribute.
`<reference>`	External citation wrapping an `<a href="...">...</a>` link record.
`<logic>`	Strong Kleene operator (`<and>`, `<or>`, `<not>`, `<non>`) over other truth entries. Wraps one operator child wit
`<provider>`	External LLM provider definition.
`<authority>`	Pointer to a remote TruthSet.

Example

```
<truth>
  <fact id="t_001" title="Mammals" DoT="0.9" time="2026-03-06T10:00:00Z">
    All dogs are mammals.
  </fact>
```

```
  <feeling id="t_002" title="Preference" time="2026-03-06T10:00:01Z">
    I prefer cats.
  </feeling>
  <reference id="t_003" title="Wikipedia: Dog" DoT="0.8">
    <a href="https://en.wikipedia.org/wiki/Dog">Wikipedia: Dog</a>
  </reference>
  <logic id="t_op1" title="Both mammals" DoT="0.0">
    <and><ref id="t_001"/><ref id="t_003"/></and>
  </logic>
  <provider id="p_claude" title="Claude" DoT="0.8">
    <api_url>https://api.anthropic.com/v1/messages</api_url>
    <model>claude-sonnet-4-6</model>
  </provider>
  <authority id="a_remote" title="Remote KB" DoT="0.5">
    <url>https://example.org/truth.xml</url>
    <refresh>3600</refresh>
  </authority>
</truth>
```

Internally, `xml_to_state()` still normalizes each typed truth element into the runtime representation, but on disk it remains XML:

```
<fact id="t_001" title="Mammals" DoT="0.9" time="2026-03-06T10:00:00Z">
  All dogs are mammals.
</fact>
```

That keeps the rest of the pipeline stable while the XML surface stays typed.

Static vs dynamic truth

When RAG is enabled, the TruthSet is processed in two phases:

1. `static_truth` extracts evaluable content such as facts, feelings, and references.
2. `dynamic_truth` evaluates operators, providers, and authorities against that static set.

All truth entries are still available to the final provider bundle.

Serialization

`state_to_xml(state)` serializes the internal nested tree to the XML grammar above. `xml_to_state(text)` parses it back. `atomic_write_xml(path, state)` uses a temp file, `fsync`, and `rename` for atomic writes.

Merge

`merge_llm_states(base, incoming)` merges an imported state into the current state with collision-safe deterministic ID suffixing. `merge_many_states(base, *incoming)` chains multiple merges.

In-memory representation

The internal runtime representation remains conversation-centric. It keeps the per-node `selected` flags and also derives helper fields such as `selected_conversation` and `selected_message` from persisted XML like this:

```
<state>
  <client>
    <version>2</version>
    <schema>https://raw.githubusercontent.com/arborrhythms/WikiOracle/main/data/state.xsd</schema>
    <time_creation>2026-03-07T12:00:00Z</time_creation>
```

```
    <time_lastModified>2026-03-07T12:00:00Z</time_lastModified>
    <title>My Project</title>
    <client_name>Alice</client_name>
    <client_id>a1b2c3d4-...</client_id>
    <ui>
      <layout>horizontal</layout>
      <theme>system</theme>
      <model></model>
      <divider_pos>0</divider_pos>
      <swipe_nav_horizontal>true</swipe_nav_horizontal>
      <swipe_nav_vertical>false</swipe_nav_vertical>
      <confirm_actions>false</confirm_actions>
    </ui>
  </client>
  <conversation id="c_root" selected="true">
    <title>Animals</title>
    <message id="m1" role="user" username="Alice" time="..." selected="true">
      <content><Q>...</Q></content>
    </message>
    <conversation id="c_dogs" parentId="c_root">
      <title>Dog breeds</title>
    </conversation>
  </conversation>
  <truth>
    <fact id="t_001" title="Mammals" DoT="0.9">
      All dogs are mammals.
    </fact>
  </truth>
</state>
```

At runtime, `bin/state.py` normalizes that XML into conversation nodes with `messages[]` and `children[]`, and derives helpers such as `selected_conversation="c_root"` and `selected_message="m1"`.

Schema

The state schema is (`data/state.xsd`)[state.xsd].

Data model

On disk – XML

State is persisted as XML (WikiOracle State format, validated by `data/state.xsd`). The XML surface is:

```
<?xml version="1.0" encoding="UTF-8"?>
<state>
  <client>
    <version>2</version>
    <schema>https://raw.githubusercontent.com/arborrhythms/WikiOracle/main/data/state.xsd</schema>
    <time_creation>2026-03-05T12:00:00Z</time_creation>
    <time_lastModified>2026-03-05T12:00:00Z</time_lastModified>
    <title>My Project</title>
    <client_name>Alice</client_name>
    <client_id>a1b2c3d4-...</client_id>
  </client>
  <conversation id="c_abc">
    <title>Animals</title>
```

```
    <message id="m1" role="user" username="Alice" time="...">
      <content><Q><fact trust="0.5">Dogs are mammals.</fact></Q></content>
    </message>
    <conversation id="c_def" parentId="c_abc">
      <title>Dogs</title>
      <message id="m2" role="assistant" username="claude" time="...">
        <content><R><fact trust="0.9">Dogs are mammals.</fact></R></content>
      </message>
    </conversation>
  </conversation>
  <truth>
    <fact id="t_001" title="Mammals" DoT="0.9" time="...">
      All dogs are mammals.
    </fact>
  </truth>
</state>
```

In memory – nested tree

On load, conversations are normalized into a nested tree in memory. The same shape in XML is:

```
<conversations>
  <conversation id="c_abc">
    <title>Animals</title>
    <messages />
    <conversations>
      <conversation id="c_def">
        <title>Dogs</title>
        <messages />
        <conversations />
      </conversation>
      <conversation id="c_ghi">
        <title>Cats</title>
        <messages />
        <conversations />
      </conversation>
    </conversations>
  </conversation>
</conversations>
```

Each **conversation** has: `id`, `title`, zero or more messages, and zero or more child conversations.

Each **message** has: `id`, `role` (user | assistant | system), `username`, `timestamp`, `content` (XHTML).

Grammar

```
State        $\rightarrow$ Client + Conversation* + Truth?
Conversation $\rightarrow$ title + (Message | Conversation)*
Message      $\rightarrow$ <message id="" role="" username="" time=""><content>...</content></message>
Truth        $\rightarrow$ Fact | Feeling | Reference | Logic | Provider | Authority
```

In memory, `bin/state.py` still normalizes conversations into `messages[]` and `children[]`.

User Interface

Rendering

The display layer covers HTML structure, CSS classes, D3 node shapes, optimistic UI, and state persistence.

Context for the upstream LLM

When a message is sent, the server needs to provide conversational context to the upstream model. The function `get_context_messages(conversations, conv_id)` walks the **ancestor chain** from the active conversation up to the root, collecting all messages in chronological order.

```
Root conversation      ->  messages: [m1, m2, m3]
  |- Child conv        ->  messages: [m4, m5]
     `- Grandchild     ->  messages: [m6, m7]  <- active

Context sent to LLM: [m1, m2, m3, m4, m5, m6, m7]
```

This gives the LLM the full path of dialogue that led to the current point, without noise from sibling branches.

Tree visualisation (D3)

`tree.js` renders the conversation tree as a top-down hierarchy using `d3.tree()`:

```
conversationsToHierarchy(state.conversations, selectedId)
    $\downarrow$
D3 hierarchy data  { id, title, messageCount, questionCount, selected, children }
    $\downarrow$
renderTree(hierarchyData, callbacks)
    $\downarrow$
SVG with nodes (rects/pills/circles) + curved links
```

The root node is a circle labelled `/`. Selected conversations render as larger labelled rectangles; others as compact pills. The selected node is highlighted with the accent colour.

Chat view

`wikioracle.js $\rightarrow$ renderMessages()` displays the messages of the currently selected conversation:

1. Look up `selectedConvId` in the tree via `findConversation()`
2. Iterate `conv.messages`, rendering each as a `.message` div with role-based styling
3. Attach drag-to-reorder (HTML5 drag/drop) and right-click context menus to each message
4. Re-render the D3 tree in sync

When no conversation is selected (root view), a placeholder prompts the user to type.

Rendering pipeline (end to end)

```
state.xml on disk
    $\downarrow$  [xml_to_state]
In-memory state with nested conversation tree
    $\downarrow$  [GET /state]
Client receives state payload
    $\downarrow$  [renderMessages]
Chat panel shows selected conversation's messages
    $\downarrow$  [conversationsToHierarchy]
D3 hierarchy data
    $\downarrow$  [renderTree]
```

`SVG tree visualisation`

Interactions

Tree panel

Gesture	Action
Click	Navigate – select that conversation, show its messages
Double-click	Open context menu (Branch, Delete)
Right-click	Open context menu (same)
Drag node → drop on node	Merge – append source's messages into target, remove source

A 200ms timer disambiguates click from double-click. Context menus are appended to `document.body` with fixed positioning to avoid clipping by the tree container's `overflow: hidden`.

Keyboard navigation

Key	Action
Arrow Up	Navigate to parent conversation
Arrow Down	Navigate to first child conversation
Arrow Right	Next node in inorder traversal
Arrow Left	Previous node in inorder traversal

Keyboard events are ignored when a text input, textarea, or select element has focus, or when the context overlay is active.

Diamond traversal A diamond (DAG merge) occurs when a conversation has multiple parents (e.g. the final synthesis in an HME vote that merges two beta branches):

```
      root
     /    \
  beta1  beta2
     \    /
      final      $\leftarrow$ parentId: [beta1, beta2]
```

Arrow Right / Arrow Left perform standard tree preorder traversal. In a DAG the same node appears under multiple parents; it is visited **once per parent** so that each parent's subtree is a complete reading path:

```
$\rightarrow$  root  $\rightarrow$  beta1  $\rightarrow$  final  $\rightarrow$  beta2  $\rightarrow$  fi
```

Chat panel

Gesture	Action
Right-click message	Context menu (Move up, Move down, Delete)
Drag message	Reorder within the conversation

Merge semantics

Dragging conversation A onto B:

1. Appends A's messages after B's existing messages (preserving order)
2. Re-parents A's children under B
3. Removes A from the tree

Branching

Double-click or right-click a tree node → "Branch" creates a new empty child conversation. The next message typed seeds it. The LLM receives the full ancestor context.

Settings Dialog

The Settings panel (gear icon in the sidebar) lets users configure evaluation, truthset, and training parameters at runtime. Settings are persisted in `sessionStorage` (with a `localStorage` fallback for tab-close durability) under the `wikioracle_config` key; UI preferences (layout, theme, confirm actions) are also saved in state.

Settings Controls

Control	DOM id	Type	Stored at	Default /
Username	`setUsername`	text	`state.client_name`	`"User"`
Provider	`setProvider`	select	`config.providers.default`	`"wikiora`
Model	`setModel`	select	`state.ui.model`	provider r
Provider trust	`setProviderTrust`	range (0-1)	`config.server.providers.<selected>.trust`	`1.0` for `wi`
Layout	`setLayout`	select	`state.ui.layout`	saved valu
Theme	`setTheme`	select	`state.ui.theme`	`"system"`
Temperature	`setTemp`	range (0-2)	`config.server.evaluation.temperature`	`0.7`
Allow URL fetching	`setUrlFetch`	checkbox	`config.server.evaluation.url_fetch`	`false`
Thought-free mode	`setThoughtFree`	checkbox	`config.server.evaluation.thought_free`	`false`
Confirm deletes / merges	`setConfirm`	checkbox	`state.ui.confirm_actions`	`false`

Settings Persistence

Setting family	Authority
`config.server.truthset`, `config.server.evaluation`, `config.server.training`, `config.providers.*`	server/config
`config.server.providers`	runtime meta
`state.client_name`, `state.ui.*`	client/state

Truth Weight Slider

The **truth weight** slider (0.0-1.0, step 0.05) replaces the former "Use TruthSet" checkbox. It controls two things simultaneously:

1. **RAG delivery**: When `truth_weight > 0`, the TruthSet is sent to the provider as grounding evidence (the former `rag: true` behavior). When `truth_weight = 0`, no truth is sent.
2. **Training modulation**: During online training, `truth_weight` controls how much DegreeOfTruth gates the learning rate:
   - `truth_weight = 0.0`: Vanilla SFT – trains on everything at full learning rate regardless of DoT. No EMA anchor pull.
   - `truth_weight = 0.7`: Default – DoT significantly gates learning, with moderate anchor pull toward checkpoint.
   - `truth_weight = 1.0`: Fully DoT-gated – zero DoT means zero learning. Full anchor pull toward checkpoint.

The slider displays its current numeric value next to the control via the `#setTruthWeightVal` span, updated live on input events.

Max Truth Entries

Controls the maximum size of the server TruthSet. When the table exceeds this limit, entries with `|trust|` closest to 0.0 (lowest information value) are trimmed during the merge stage. Range: 100-10000, step 100.

Store Concrete Facts

When enabled, spatiotemporally-bound facts (those with `<place>` or `<time>` child elements carrying real values) are stored in the server TruthSet alongside universal facts. When disabled (default), only universal facts persist – consistent with Zero-Knowledge / Selective Disclosure principles.

This setting is a client-side override for the server's `store_concrete` config. The client value takes precedence when sent in the query payload.

Legacy Migration

The config system automatically migrates the former boolean `rag` flag to the new `truth_weight`:

- `rag: true` → `truth_weight: 0.7`
- `rag: false` → `truth_weight: 0.0`

This migration runs in both `client/config.js` (client-side) and `bin/response.py` (server-side) to handle configs from before the migration. The new config path for truth weight is `server.truthset.truth_weight`.

Server TruthSet Display

In debug mode, when online training is enabled, the client displays server TruthSet entries alongside local truth entries. Server entries are visually distinguished with:

- A blue left border (3px, accent colour)
- A "SERVER" badge in small uppercase text
- Reduced opacity (0.65)

Server truth entries are injected from the `/chat` response's `server_truth` field (debug mode only) and deduplicated by `id` on each refresh.

User Interface Strings

Canonical UI text for the WikiOracle client. A consistency test (`test/test_ui_strings.py`) periodically checks that the strings hard-coded in `client/util.js` match the tables below.

Truth editor

Dropdown labels

Key	Label
`feeling`	`Feeling: subjective, non-verifiable claim`
`fact`	`Fact: disprovable assertion about the world`
`reference`	`Reference: citation with external link`
`authority`	`Authority: pointer to remote TruthSet`
`provider`	`Provider: external LLM endpoint`

Descriptions

Key	Description
feeling	Feeling -- a subjective, non-verifiable claim (not penalizable).
fact	Fact -- a disprovable assertion with a degree of truth.
reference	Reference -- a citation linking to an external source.
and	AND -- true when all children are true (min trust).
or	OR -- true when any child is true (max trust).
not	NOT -- negation of a child entry.
non	NON -- non-affirming negation (weakens trust toward zero).
provider	Provider -- an LLM API endpoint.
authority	Authority -- a remote knowledge base (XML URL).

Templates

Key	Template
feeling	<feeling>Subjective statement here.</feeling>
fact	<fact trust="0.0">Assertion text here.</fact>
reference	<reference trust="0.0"><a href="https://example.com">Link text</a></reference>
and	<logic><and><ref id=""/><ref id=""/></and></logic>
or	<logic><or><ref id=""/><ref id=""/></or></logic>
not	<logic><not><ref id=""/></not></logic>
non	<logic><non><ref id=""/></non></logic>
provider	<provider trust="0.0"><api_url>https://api.example.com</api_url><model>model_name</model></p
authority	<authority trust="0.0"><url>https://example.com/state.zip</url><key>password</key></authorit

Empty state

Key	Text
truth_empty	No truth entries. Add truth here or Open a new state file.

Proposed License

WikiOracle Licensing Architecture

This document proposes a licensing structure for the WikiOracle project designed to preserve software freedom, maintain an open knowledge commons, and enable responsible AI development.

Four-Layer Conceptual Model

WikiOracle separates artifacts into distinct licensing domains.

Layer Artifact Type Recommended License

Layer 1 Software / engine code GPL-3.0 Layer 2 Human knowledge content CC BY-SA 4.0 Layer 3 Structured data CC0 Layer 3B Model weights Apache-2.0

Conceptual summary:

```
code $\rightarrow$ algorithms
data $\rightarrow$ facts
weights $\rightarrow$ learned structure
content $\rightarrow$ explanations
```

This layered separation mirrors successful open knowledge systems while extending them for AI systems.

License Comparison Table

License	Domain	Key Features	Typical
GPL-3.0	Software	Strong copyleft, patent protection, source distribution requirement	Infrastru
CC BY-SA 4.0	Creative works	Attribution required, derivatives must remain under same license	Wikipec
Apache-2.0	Software / model artifacts	Permissive reuse with patent grant	AI mod

GPL-3.0

Properties:

- strong copyleft protection
- requires publication of source code
- includes explicit patent protection
- prevents proprietary forks of infrastructure

CC BY-SA 4.0

Properties:

- allows copying and adaptation
- requires attribution
- derivatives must remain under the same license
- widely used for collaborative knowledge projects

Apache-2.0

Properties:

- permissive reuse
- explicit patent protection
- compatible with commercial ecosystems
- widely used for machine learning artifacts

Rationale for Layered Licensing

Software (GPL-3.0)

The WikiOracle engine should remain permanently open. GPL ensures that improvements to the infrastructure remain open-source and cannot be captured by proprietary forks.

Knowledge Content (CC BY-SA)

Human-authored knowledge should remain part of a global commons. CC BY-SA ensures attribution and share-alike behavior so improvements remain public.

Structured Data (CC0)

Structured datasets (truth graphs, semantic relations, dataset exports) benefit from minimal legal friction. CC0 allows unrestricted reuse and avoids attribution chains that complicate machine learning workflows.

Model Weights (Apache-2.0)

Model weights are derived artifacts produced by training. Apache-2.0 allows redistribution, fine-tuning, and commercial use while providing patent protection.

Training Data Provenance Policy

AI systems increasingly require transparent documentation of training sources.

WikiOracle should maintain a provenance record including:

- dataset origin
- license compatibility
- ingestion date
- preprocessing steps
- contributor identity (if applicable)

This allows:

- legal defensibility
- reproducibility
- verification of training inputs

Example provenance record structure:

```
dataset_name
source_url
license
date_ingested
transformation_pipeline
```

Provenance tracking supports both transparency and scientific reproducibility.

AI-Generated Content Policy

AI-generated text occupies a hybrid authorship category.

WikiOracle should treat generated content in three stages:

Stage License

Raw AI output CC0 AI draft edited by human CC BY-SA Human-authored content CC BY-SA

Rationale:

- raw AI output may lack clear authorship
- human editorial involvement establishes attribution
- share-alike preserves the knowledge commons

All AI contributions should record metadata:

```
generation_model
timestamp
human_editor
revision_history
```

This ensures transparency regarding machine involvement.

Summary

WikiOracle licensing stack:

```
Layer 1 -- Software
GPL-3.0

Layer 2 -- Knowledge Content
```

```
CC BY-SA 4.0

Layer 3 -- Structured Data
CC0

Layer 3B -- Model Weights
Apache-2.0
```

The architecture protects the software commons, maintains a global knowledge commons, and supports open AI development.